Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

March 9, 2026

1 Home
2 Open-FDD
  2.1 What it does
  2.2 Quick start
    2.2.1 Protecting the entire API with Caddy
  2.3 Documentation
  2.4 Stack
  2.5 Behind the firewall; cloud export is vendor-led
3 System Overview
4 System Overview
  4.1 Architecture
  4.2 Services
  4.3 Campus-based architecture
  4.4 Data flow
  4.5 Ways to deploy
  4.6 Key concepts
5 Getting Started
6 Getting Started
  6.1 Do this to bootstrap
  6.2 Prerequisites
  6.3 What the bootstrap script does
  6.4 After bootstrap
  6.5 Next steps
7 Using the React dashboard
8 Using the React dashboard
  8.1 Navigation
  8.2 Common workflows
9 BACnet Overview
10 BACnet Integration
  10.1 Components
  10.2 Ports

10.3 Discovery and getting points into the data model
10.4 Data acquisition
10.5 Configuration
11 BACnet, RDF, and BRICK
12 BACnet, RDF, and BRICK
12.1 Roles
12.2 Flow
12.3 Try it (Open-FDD Swagger)
12.4 Summary
13 BACnet
14 BACnet
14.1 Setup (do this before the platform scrapes data)
15 Overview
16 Data model flow
16.1 Flow
16.2 TTL structure
16.3 Data-model API
16.4 Validation
17 Sites
18 Sites
18.1 Structure
18.2 Scoping
18.3 API
19 Equipment
20 Equipment
20.1 Structure
20.2 Hierarchy
20.3 API
21 AI-assisted data modeling
22 AI-assisted Brick tagging
22.1 Workflow (export → tag → import)
22.2 API contract (import)
22.3 LLM prompt and agent guidelines
22.4 See also
23 Points
24 Points
24.1 Structure
24.2 Time-series data
24.3 Layers and mapping
24.3.1 BACnet addressing
24.4 API
24.5 Queries
25 SPARQL cookbook

26 SPARQL cookbook
26.1 How to run SPARQL
26.2 Prefixes and graph contents
26.3 Recipe 1: Platform config
26.4 Recipe 2: Sites and counts
26.5 Recipe 3: Equipment and points by site
26.6 Recipe 4: BACnet devices and objects
26.7 Recipe 5: FDD — points and rule mapping
26.8 Recipe 6: FDD — time-series references for rules
26.9 Recipe 7: Integrity and completeness
26.10 E2E test coverage (graph_and_crud_test.py)
26.11 Saved queries and automated tests
26.12 Related docs
27 Data modeling
28 Data modeling
29 Overview
30 Fault rules
30.1 Where to place your rules
30.2 Hot reload (edit → run → view in Grafana)
30.3 Rule types
30.4 YAML structure
30.5 Input resolution (100% Brick)
30.6 Running rules
30.7 Cookbook
31 Expression Rule Cookbook
32 Expression Rule Cookbook
32.1 100% Brick-model driven (no column in rules)
32.2 How to define expressions
32.3 AHU rules (reference-style)
32.3.1 Rule A — Duct static below setpoint at full fan speed
32.3.2 Rule B — Blended air temp below expected band
32.3.3 Rule C — Blended air temp above expected band
32.3.4 Rule D — Discharge air cold when heating commanded
32.3.5 Rule E — SAT too low with full heating
32.3.6 Rule F — SAT/MAT mismatch in economizer mode
32.3.7 Rule G — Ambient too warm for free cooling
32.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)
32.3.9 Rule I — Ambient vs blended mismatch (econ-only)
32.3.10 Rule J — Discharge above blended in cooling
32.3.11 Rule K — Discharge above setpoint in full cooling
32.3.12 Rule L — Cooling coil delta-T when inactive
32.3.13 Rule M — Heating coil delta-T when inactive

32.4 Central plant
32.4.1 Differential pressure at max pump speed
32.4.2 Plant flow high at max pump
32.4.3 Plant supply temp outside deadband
32.4.4 Chiller runtime over daily limit
32.5 Heat pumps
32.5.1 Discharge cold when heating
32.6 VAV zones
32.6.1 Excessive heating during warm weather
32.6.2 Damper or valve at full open
32.6.3 Zone and IAQ bounds
32.7 Opportunistic rules (economizer & ventilation)
32.7.1 Economizing when outdoor conditions are unfavorable
32.7.2 Mechanical cooling when econ could suffice
32.7.3 Low ventilation (estimated OA fraction)
32.7.4 Preheat over-conditioning
32.7.5 Blended air damper deviation
32.8 Weather station
32.8.1 Unrealistic temperature spike
32.8.2 RH bounds
32.8.3 Wind gust vs sustained
32.9 Sensor validation (bounds & flatline)
33 Fault rules for HVAC
34 Fault rules for HVAC
35 Cloud export example
36 Cloud export example
36.1 What the example does
36.2 Run the example
36.3 Use it as your starting point
37 Concepts
38 Concepts
39 How-to Guides
40 How-to Guides
41 Grafana SQL cookbook
42 Grafana SQL cookbook —
42.1 BACnet
42.1.1 1. Site Variable
42.1.2 2. Device Variable
42.1.3 3. Point Variable
42.1.4 BACnet only data
42.2 BACnet Plus Fault Data
42.3 Host stats (stack-host-stats)
42.3.1 Dashboard variable (Host)
42.3.2 Host — Memory (used / available / total)

42.3.3 Host — Load (1 / 5 / 15 min)
42.3.4 Host — Swap (used / total)
42.3.5 Host — Disk space (free / used / total)
42.3.6 Host — Disk available (gauge, latest)
42.3.7 Containers — CPU %
42.3.8 Containers — Memory %
42.3.9 Containers — Table (latest)
42.4 Weather
42.4.1 Temp RH Dewpoint
42.4.2 Wind Speed Gusts Direction
42.4.3 Wind Speed Gust Direction
42.4.4 Solar Radiation
42.4.5 Cloud Cover
42.5 Faults
42.5.1 Fault Definitions
42.5.2 Fault Definition Count
43 Grafana dashboards
44 Grafana dashboards (optional)
44.1 React frontend parity
45 Configuration
46 Configuration
46.1 Platform config (RDF + CRUD)
46.2 Platform keys (RDF / GET-PUT /config)
46.3 Services that read config from the API (BACnet scraper)
46.4 BACnet: single gateway, remote gateways, central aggregator
46.5 Edge / resource limits
46.6 Environment
47 Contributing
48 Contributing to Open-FDD
48.1 Table of Contents
48.2 Code of Conduct
48.3 I Have a Question
48.4 I Want To Contribute
48.4.1 Legal notice
48.5 Reporting Bugs
48.5.1 Before submitting a bug report
48.5.2 How to submit a good bug report
48.6 Suggesting Enhancements
48.6.1 Before submitting
48.6.2 How to submit a good enhancement suggestion
48.7 Contributing Rules (Expression Rule Cookbook)
48.8 Your First Code Contribution
48.9 Improving the Documentation
48.10 Styleguides

48.11 E2E frontend tests (Selenium)
48.12 Git workflow (branch → PR → sync)
48.13 Commit Messages
48.14 Join the Project Team
48.15 Attribution
49 API Reference
50 API Reference
50.1 High-level overview
51 Appendix
52 Appendix
53 Frontend Evaluation
54 React frontend evaluation
54.1 Quality and stack
54.2 Feature coverage vs OpenAPI CRUD
54.3 Grafana cookbook parity
54.4 WebSockets: keep or remove?
54.5 Legacy static UI and BACnet tree
54.6 Caddy and security
54.7 Summary
55 Security
56 Security and Caddy Bootstrap
56.1 Architecture: frontend, API, and Caddy
56.2 Frontend API key (Bearer)
56.3 Quick bootstrap with Caddy and basic auth
56.4 Caddyfile for protecting the entire API
56.5 Default password and how to change it
56.6 Services: encryption and exposure
56.7 Hardening best practices
56.8 Throttling and rate limiting
56.8.1 1. No API rate limiting by default
56.8.2 2. Outbound: OT/building network is paced
56.8.3 3. Inbound: rate limiting at the reverse proxy (e.g. Caddy)
56.9 TLS (optional): self-signed certificates
56.10 Troubleshooting
56.11 Security scanning (Trivy) — Phase 3
56.12 Summary

Generated March 9, 2026

---

1 Home

2 Open-FDD

Open-FDD is an **open-source knowledge graph for building technology systems**, specializing in **fault detection and diagnostics (FDD) for HVAC**. It runs **on-premises** so facilities keep control of data and avoid vendor lock-in; DoE research reports median energy savings of ~8–9% from FDD programs. The platform is an AFDD stack that transforms operational data into actionable insights and provides a secure integration layer for cloud vendors without lock-in.

At its core is a **unified graph**: one semantic model that combines Brick (sites, equipment, points), BACnet discovery RDF, platform config, and —as the project evolves—other ontologies such as ASHRAE 223P. That single graph is queried via SPARQL and serialized to `config/data_model.ttl`; CRUD and discovery both update it.

2.1 What it does

Open-FDD is an **edge analytics and rules engine** for building automation. It:

- **Ingests** BACnet points via diy-bacnet-server (JSON-RPC) and weather via Open-Meteo
- **Stores** telemetry in TimescaleDB and models the building in a **unified RDF graph** (Brick + BACnet + config)
- **Runs** YAML-defined FDD rules (bounds, flatline, hunting, expression) on a configurable schedule
- **Exposes** REST APIs for sites, points, equipment, data-model export/import, bulk timeseries and fault download (Excel-friendly CSV, JSON for cloud), TTL generation, SPARQL validation
- **Visualizes** timeseries and fault results in Grafana

Operators and integrators get full control, lower cost, and no vendor lock-in. Already powering HVAC optimization and commissioning workflows.

2.2 Quick start

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
./scripts/bootstrap.sh
```

Endpoints (direct access):

What	URL	Notes
Frontend (React)	http://localhost:5173	Main UI: dashboard, sites, points, config, faults, plots. Use this for day-to-day workflows.
API (REST)	http://localhost:8000/docs	Swagger UI for integration and scripts. High-level reference: Appendix: API Reference.
Caddy (reverse proxy)	http://localhost:80	Default `stack/caddy/Caddyfile` proxies to frontend only. See Protecting the entire API with Caddy below.
TimescaleDB	localhost:5432	Database `openfdd` (user `postgres`); keep internal.
BACnet (diy-bacnet-server)	http://localhost:8080/docs	JSON-RPC API; UDP 47808 for BACnet/IP.
Grafana	http://localhost:3000	**Optional:** `./scripts/bootstrap.sh --with-grafana` (admin/admin). React frontend provides equivalent views.

WebSockets: The API exposes **`/ws/events`** for live updates (faults, CRUD, FDD run). The React frontend connects with `?token=<API_KEY>` when `VITE_OFDD_API_KEY` is set; Bearer auth is used for REST. See Security & Caddy.

2.2.1 Protecting the entire API with Caddy

The default **`stack/caddy/Caddyfile`** only routes **port 80 → frontend** (no auth, API not behind Caddy). To protect the **entire** stack behind one entry point (basic auth, API + WebSocket + frontend, optional Grafana):

1. **Use a Caddyfile that:**
   - Listens on one port (e.g. `:80` or `:8088`).
   - Enables **basic_auth** for all routes (one login for the browser).
   - Proxies **`/api/*`**, **`/ws/*`**, **`/docs`**, **`/redoc`**, **`/openapi.json`**, **`/health`**, and other API paths to **`api:8000`**.
   - Proxies **`/grafana`** to **`grafana:3000`** if you use `--with-grafana`.
   - Proxies `/*` to the **frontend** (e.g. `frontend:5173`).
   - When proxying to the API, adds **`header_up X-Caddy-Auth <secret>`** so the API accepts requests that passed basic auth (set `OFDD_CADDY_INTERNAL_SECRET` in the API container to the same value).
2. **Full steps:** See Security and Caddy — Quick bootstrap with Caddy and basic auth, default password change, and troubleshooting (401s, WebSocket behind Caddy).
3. **Caddyfile location:** `stack/caddy/Caddyfile`. A full example (basic auth, API, WebSocket, frontend) is in Security — Caddyfile for protecting the entire API.

2.3 Documentation

Section	Description
System Overview	Architecture, services, data flow
Getting Started	Install, bootstrap, first run
Using the React dashboard	Overview, Config, Points, Data model, Faults, Plots, Web weather, System resources
BACnet	Discovery, scraping, RPC, RDF/BRICK (knowledge graph, bacpypes3)
Data modeling	Sites, equipment, points (CRUD), Brick TTL, SPARQL cookbook, AI-assisted tagging
Fault rules for HVAC	Rule types, expression cookbook

Section	Description
Concepts	Cloud export example — how vendors pull data from the API to their cloud
How-to Guides	Grafana dashboards (optional), Grafana SQL cookbook
Security & Caddy	Basic auth, bootstrap, hardening, optional TLS
Configuration	Platform config, rule YAML
Appendix	API Reference — REST at a glance; Technical reference, Developer guide
Contributing	How to contribute; alpha/beta focus; bugs, rules, docs, drivers

Use the React frontend for config, sites, points, data model, faults, and plots. **API details** (CRUD, config, data-model, download, analytics, BACnet) are summarized in Appendix: API Reference; full OpenAPI at `/docs` and `/openapi.json`. **Grafana** is optional; the React UI provides equivalent timeseries and fault views.

2.4 Stack

Service	Port	Purpose
Caddy	80	Reverse proxy (default: frontend only; see Security to protect API + WebSocket + frontend)
API	8000	REST API, Swagger `/docs`, WebSocket `/ws/events`
Frontend	5173	React dashboard (sites, points, faults, plots)
TimescaleDB	5432	PostgreSQL + TimescaleDB
Grafana	3000	**Optional** dashboards (`--with-grafana`); React frontend has equivalent views
diy-bacnet-server	8080	JSON-RPC API (HTTP); POST server_hello

Service	Port	Purpose
		returns `mqtt_bridge` when BACnet2MQTT enabled
diy-bacnet-server	47808	BACnet/IP (UDP)
Mosquitto (MQTT)	1883	Optional: `./scripts/bootstrap.sh --with-mqtt-bridge` — broker for BACnet2MQTT

2.5 Behind the firewall; cloud export is vendor-led

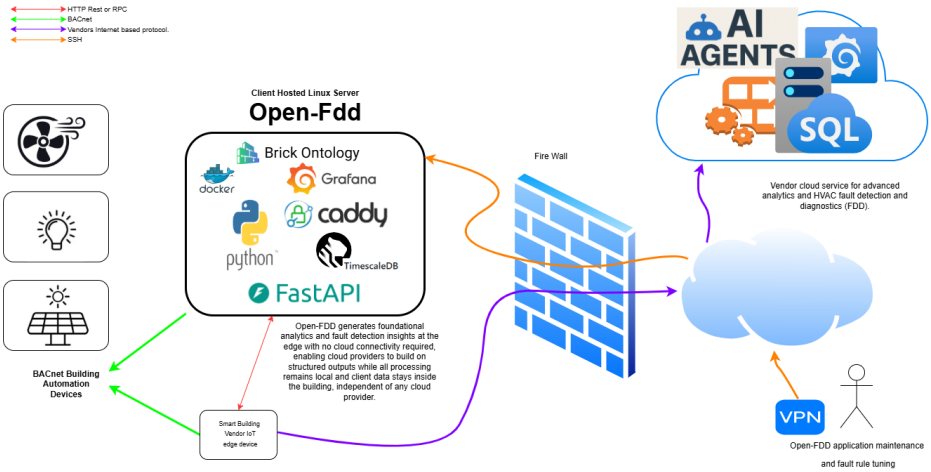
Open-FDD runs **inside the building, behind the firewall**. Analytics and rules run locally; telemetry stays on-premises unless you export it. Cloud/MSI vendors run their **own** export process (e.g. edge gateway or script) that **pulls** from the API over the LAN; Open-FDD does not push to the cloud. See the Cloud export example and `examples/cloud_export.py`. Data stays with the client; the open data model and APIs let you switch vendors without losing data or the model.

3 System Overview

4 System Overview

Open-FDD is an **open-source knowledge graph for building technology systems**, specializing in fault detection and diagnostics (FDD) for HVAC. FDD is often called AFDD when the platform runs it on a schedule (automated FDD). It runs on-premises so facilities keep control of their data and avoid vendor lock-in. This section describes the architecture, services, and data flow.

4.1 Architecture



Open-FDD Edge Platform Architecture

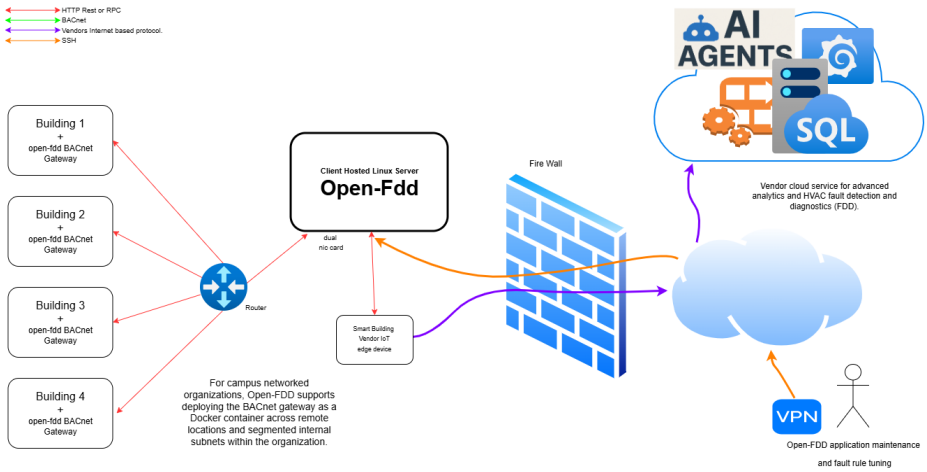
This project is an open-source stack; a cloud or MSI vendor can develop their own Docker container and deploy it on the **same client-hosted server** that runs Open-FDD, pulling from the local API over the LAN. That approach removes the need for a separate IoT or edge device dedicated to the vendor.

4.2 Services

Service	Description
API	FastAPI CRUD for sites, equipment, points. Data-model export/import, TTL generation, SPARQL validation. Swagger at `/docs`. Config UI (HA-style data model tree, BACnet test) at `/app/`.
Grafana	Pre-provisioned TimescaleDB datasource only (uid: openfdd_timescale). No dashboards; build your own with SQL from the Grafana SQL cookbook. Use `--reset-grafana` to re-apply datasource provisioning.
TimescaleDB	PostgreSQL with TimescaleDB extension. Single source of truth for metadata and time-series.
BACnet scraper	

Service	Description
	Polls diy-bacnet-server via JSON-RPC. Writes readings to `timeseries_readings`.
Weather scraper	Fetches from Open-Meteo ERA5 (temp, RH, dewpoint, wind, solar/radiation, cloud cover).
FDD loop	Runs every N hours (see `rule_interval_hours`, `lookback_days` in platform config). Pulls last N days from DB into pandas, **reloads all rules from YAML on every run** (hot reload), runs rules, writes `fault_results` back to DB. No restart needed when tuning rule params.
diy-bacnet-server	BACnet/IP JSON-RPC bridge. Discovered devices/points → CSV; scraper reads present-value via RPC.

4.3 Campus-based architecture



Open-FDD Edge Platform Architecture Campus

Remote Open-FDD BACnet gateways (e.g. diy-bacnet-server plus scraper) can be deployed **across each subnet** on the internal campus IT network. Typically each building has its own BACnet network on a unique subnet; a gateway per building or per subnet keeps BACnet traffic local while forwarding data to a **centralized** Open-FDD instance (API, Grafana, FDD loop, database). That gives the campus a single

integration point for the cloud-based vendor of choice—one API and one data model for the whole portfolio, without the vendor touching each building's BACnet network directly.

How to set it up: (1) **Remote gateway per building:** On each subnet run diy-bacnet-server + scraper; set the scraper's `OFDD_DB_DSN` to the central database and `OFDD_BACNET_SITE_ID` to that building's site (create the site on the central API first). (2) **Central aggregator:** On the central host run only DB, API, Grafana, FDD loop (no local BACnet containers); set `OFDD_BACNET_GATEWAYS` to a JSON array of `{url, site_id, config_csv}` and run one scraper that polls each remote gateway. See Configuration — BACnet for keys and examples.

4.4 Data flow

1. **Ingestion:** BACnet scraper and weather scraper write to `timeseries_readings` (point_id, ts, value).
2. **Data model (unified graph):** The building is represented as a **unified graph**—one semantic model combining Brick (sites, equipment, points from the DB), BACnet RDF (from point discovery via diy-bacnet-server), platform config, and room for future ontologies (e.g. ASHRAE 223P). CRUD and **POST /bacnet/point_discovery_to_graph** update this model; SPARQL queries it. One TTL file `config/data_model.ttl` holds the graph; a background thread serializes it to disk every 5 minutes (configurable via `OFDD_GRAPH_SYNC_INTERVAL_MIN`); **POST /data-model/serialize** runs the same write on demand.
3. **FDD (Python/pandas):** The FDD loop pulls data into a pandas DataFrame, runs YAML rules, writes `fault_results` to the database. Fault logic lives in the rule runner; the database is read/write storage.
4. **Visualization:** Grafana queries TimescaleDB for timeseries and fault results.

4.5 Ways to deploy

- **Docker Compose** (recommended): `./scripts/bootstrap.sh`
- **Minimal (raw BACnet only):** DB + Grafana + BACnet server + scraper: `./scripts/bootstrap.sh --minimal` — no FDD, weather, or API.

- **Manual:** Start DB, Grafana, run scrapers and FDD loop from host

---

4.6 Key concepts

- **Sites** — Buildings or facilities.
- **Equipment** — Devices (AHUs, VAVs, heat pumps). Belong to a site.
- **Points** — Time-series references. Have `external_id` (raw name), `rule_input` (FDD column ref), optional `brick_type` (Brick class).
- **Fault rules** — YAML files (bounds, flatline, hunting, expression). Run against DataFrame; produce boolean fault flags. See Fault rules for HVAC.
- **Unified graph** — One semantic model (Brick + BACnet + platform config; future 223P or other ontologies). Stored in `config/data_model.ttl`; maps Brick classes → `external_id` for rule resolution; queryable via SPARQL.

5 Getting Started

6 Getting Started

This page covers **prerequisites** and the **bootstrap script**: how to get the Open-FDD platform running. For configuration, data modeling, and rules, see the Documentation index.

6.1 Do this to bootstrap

1. **Prerequisites:** Docker, Docker Compose, and Git installed (see below).

2. **Clone and run bootstrap** (from any directory):

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
./scripts/bootstrap.sh
```

That's it. The script builds and starts the full stack (DB, API, frontend, Caddy, diy-bacnet-server, BACnet scraper, weather scraper, FDD loop), waits for Postgres, runs migrations, and **seeds platform config** via the API (PUT /config) so runtime settings are in the knowledge graph. When it finishes you get:

- **API:** http://localhost:8000/docs
- **Frontend:** http://localhost:5173 (or via Caddy http://localhost:80). See Using the React dashboard for what each page does.
- **BACnet Swagger:** http://localhost:8080/docs
- **DB:** localhost:5432/openfdd (postgres/postgres)
- **Grafana:** not started by default; use `./scripts/bootstrap.sh --with-grafana` then http://localhost:3000 (admin/admin)

3. **Optional:** Set `OFDD_*` in `stack/.env` before the first run to customize the seeded config (e.g. `OFDD_BACNET_SERVER_URL`, `OFDD_RULE_INTERVAL_HOURS`). See Configuration.

6.2 Prerequisites

- **OS:** Linux only (Ubuntu Server latest, or Linux Mint). **Tested on x86;** should work on ARM but is untested. The bootstrap script and Docker stack are not supported on Windows. Keep the system updated:

```
sudo apt update && sudo apt upgrade -y
```

- **Docker and Docker Compose:** Required. Install Docker Engine and Docker Compose (or `docker-compose`). See Docker install for your distro.

- **Git:** To clone the project:

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
```

- **BACnet (default data driver):** The default data driver is BACnet. Bootstrap **automatically** builds and starts diy-bacnet-server as its own Docker container (plus the BACnet scraper). You must run **BACnet discovery first** and curate the resulting CSV before the platform can scrape data—the scraper uses that CSV as its config. See BACnet → Setup and BACnet overview. To run without BACnet

(e.g. central-only with remote gateways), start only the services you need (e.g. `docker compose up -d db grafana api fdd-loop weather-scraper` from `stack/`).

6.3 What the bootstrap script does

`scripts/bootstrap.sh` (run from the **repo root**):

1. Ensures **diy-bacnet-server** exists as a sibling repo (clones it if missing).
2. Runs **docker compose up -d –build** from `stack/` (builds all images, starts all services).
3. Waits for **Postgres** to be ready (~15s).
4. Applies **database migrations** (idempotent; safe on existing DBs).
5. **Seeds platform config** via PUT /config (waits for API, then sends default or `stack/.env` values into the RDF graph).
6. Optionally runs **–reset-data** if you passed that flag (deletes all sites + data-model reset; for testing).

It does **not** purge or wipe the database on a normal run; only `--reset-grafana` wipes the Grafana volume.

Full stack (default): TimescaleDB, API, **diy-bacnet-server** (BACnet/IP bridge), **BACnet scraper**, weather scraper, FDD loop. Grafana is optional (`--with-grafana`). For BACnet data use the **data model** (discover via frontend or API → import points); see BACnet overview. Optional services (Caddy, host-stats) are in docker-compose; start them with `docker compose up -d` from `stack/` if needed.

Bootstrap options: Run `./scripts/bootstrap.sh --help` for the full list. Summary:

Option	Effect
(none)	Build and start full stack (DB, API, frontend, Caddy, BACnet server, scrapers, FDD loop). Prints API key if generated. Grafana **not** started by default.
`--with-grafana`	Include Grafana; then accessible at http://localhost:3000 or via Caddy at `/grafana` if using a full Caddyfile.

Option	Effect
`--minimal`	DB + BACnet server + bacnet-scraper only. No FDD, weather, or API. Add `--with-grafana` for Grafana.
`--verify`	Health checks only: list containers, test DB; exit. Does not start or stop.
`--test`	Run tests (frontend lint + typecheck, backend pytest, Caddy validate); then exit.
`--build SERVICE ...`	Rebuild and restart only listed services, then exit. Services: `api`, `bacnet-server`, `bacnet-scraper`, `caddy`, `db`, `fdd-loop`, `frontend`, `grafana`, `host-stats`, `weather-scraper`.
`--build-all`	Rebuild and restart all services; then exit.
`--frontend`	Before start: stop frontend container and remove `frontend_node_modules` volume so the next `up` runs a fresh `npm install`. Use after changing `frontend/package.json`.
`--update`	Git pull open-fdd and diy-bacnet-server (sibling), then rebuild and restart (keeps DB).
`--maintenance`	Safe Docker prune only (no volume prune).
`--reset-grafana`	Wipe Grafana volume and re-apply provisioning. DB and other data retained.
`--reset-data`	Delete all sites via API and POST /data-model/reset (testing).
`--retention-days N`	TimescaleDB retention in days (default 365). Env: `OFDD_RETENTION_DAYS`.
`--log-max-size SIZE`	

Option	Effect
	Docker log max size per file (default `100m`). Env: `OFDD_LOG_MAX_SIZE`.
`--log-max-files N`	Docker log max files per container (default 3). Env: `OFDD_LOG_MAX_FILES`.
`--install-docker`	Attempt Docker install (Linux) then continue.
`--skip-docker-install`	Explicitly skip Docker install (no-op; use with scripts that call bootstrap after install).
`--no-auth`	Do not generate or set `OFDD_API_KEY`; API will not require Bearer auth.

To **update** an existing clone: `git pull` then `./scripts/bootstrap.sh`, or `./scripts/bootstrap.sh --update`. Rebuild single services: `./scripts/bootstrap.sh --build api`.

6.4 After bootstrap

- **Grafana:** Open http://localhost:3000. TimescaleDB datasource and Open-FDD dashboards are provisioned from config. If dashboards or datasource are wrong, run `./scripts/bootstrap.sh --reset-grafana` (keeps DB data).
- **Minimal mode:** If you used `--minimal`, only DB, Grafana, BACnet server, and scraper run. No API; use Grafana and scraper logs to confirm data flow. To add the full stack later, run `./scripts/bootstrap.sh` without `--minimal`.

6.5 Next steps

- **How-to Guides** — Grafana dashboards (optional) and SQL cookbook.
- **Configuration** — Platform config, rule YAML, services that read config from the API.
- **Security & Caddy** — Basic auth, throttling, TLS.

- **Appendix: API Reference** — REST endpoints at a glance; Swagger at http://localhost:8000/docs.

For **BACnet** (discovery and data model): BACnet and BACnet overview. For data modeling and fault rules: Data modeling, Fault rules for HVAC.

7 Using the React dashboard

8 Using the React dashboard

The Open-FDD **React frontend** (http://localhost:5173 or via Caddy) is the main UI for sites, points, data model, faults, and timeseries. Use it after Getting started; ensure the API is running and (if auth is enabled) the frontend is built with `VITE_OFDD_API_KEY` or you access through Caddy with basic auth.

8.1 Navigation

Page	Purpose
Overview	High-level summary: site selector, FDD/weather status, links into the app.
OpenFDD Config	View and edit platform config (GET/PUT `/config`): FDD rule interval, lookback, BACnet, Open-Meteo, graph sync. Current settings are shown in a read-only summary; use the form to change values and Save.
Points	List points (optionally by site). Shows external ID, equipment, Brick type, FDD input, unit, polling, last value and time. Use for BACnet discovery (Who-Is, point discovery, add to data model) and CRUD.
Data model	Equipment tree, points, export/import, TTL view, SPARQL. Manage sites, equipment, and points; run data-model export/import and SPARQL.
Faults	

Page	Purpose
	Active fault states and definitions. Filter by site/equipment.
Plots	Timeseries charts. Select site, date range, and points; view sensor and weather data. Download CSV for the selected range and points.
Web weather	Open-Meteo weather charts (temp, RH, wind, radiation, etc.) when weather is enabled.
System resources	Host and container metrics (when host-stats is running): memory, load, disk, **per-container CPU/memory** (table + time-series charts). Status badges (green/yellow/red) indicate resource pressure.
Stack status	Overview shows API, BACnet gateway, and MQTT bridge status with **green** / **yellow** / **red** indicators (e.g. MQTT bridge connected = green, enabled but disconnected = yellow).

8.2 Common workflows

- **Change platform config:** OpenFDD Config → edit fields → Save. Changes take effect on the next FDD run or scraper cycle.
- **Discover BACnet points:** Points → BACnet panel (Who-Is range, then point discovery for a device) → add to data model. Points appear in the data model and Points list.
- **Inspect or edit the data model:** Data model → browse equipment and points, export/import, view TTL, run SPARQL.
- **View timeseries:** Plots → pick site and date range → select points → chart. Use Download CSV for the current selection and zoom.
- **Check faults:** Faults → see active faults and definitions; combine with Plots to correlate with sensor data.

For API integration (curl, scripts), see Appendix: API Reference and Swagger at http://localhost:8000/docs.

9 BACnet Overview

10 BACnet Integration

Open-FDD uses diy-bacnet-server as a BACnet/IP-to-JSON-RPC bridge. Discovery and scrape feed the same **data model** (building as a knowledge graph). The gateway uses **bacpypes3**'s built-in RDF (BACnetGraph) for discovery-to-RDF; Open-FDD merges that TTL and queries via SPARQL.

10.1 Components

Component	Purpose
diy-bacnet-server	BACnet/IP UDP listener + HTTP JSON-RPC API. Discovers devices and objects; exposes present-value reads. Swagger: http://localhost:8080/docs
BACnet scraper	Platform service. Polls diy-bacnet-server on a schedule; **reads points from the data model** (points with `bacnet_device_id` and `object_identifier`) and writes readings to TimescaleDB.
Data model	Sites, equipment, and points with BACnet addressing (`bacnet_device_id`, `object_identifier`, `object_name`). Configured via the **React frontend** (Config, Data model, Points) or the API. Single source of truth for what to scrape.

10.2 Ports

Port	Protocol	Use
47808	UDP	BACnet/IP
8080	HTTP	JSON-RPC API, Swagger docs

Only **one** process on the host can use port 47808 (BACnet/IP). Run discovery **before** starting diy-bacnet-server so they do not conflict. diy-bacnet-server runs with `network_mode: host` so it binds to the host's network interface.

10.3 Discovery and getting points into the data model

All BACnet configuration is done via the **data model** and the **React frontend** (or the API). There is no CSV-based scrape path.

1. **Run Who-Is and point discovery** — Use the React frontend (Config or Data model → BACnet panel) or the API. From the BACnet panel you can run **Test connection**, **Who-Is range**, and **Point discovery** (these proxy to diy-bacnet-server). diy-bacnet-server must be running (e.g. `./scripts/bootstrap.sh` starts it).
2. **Graph and data model** — Use **POST /bacnet/point_discovery_to_graph** (device instance) to put BACnet devices and points into the in-memory graph and sync `config/data_model.ttl`. Create points in the DB via the frontend or CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`), or use **GET /data-model/export** → LLM/human tagging → **PUT /data-model/import**.
3. **Run the scraper** — The BACnet scraper loads points that have `bacnet_device_id` and `object_identifier` from the database and polls only those via diy-bacnet-server.

See Points for the BACnet fields and Appendix: API Reference for endpoints. **Port:** Only one process can use port 47808 (BACnet/IP). For API/UI discovery, diy-bacnet-server is already running.

10.4 Data acquisition

The BACnet scraper loads points from the DB where `bacnet_device_id` and `object_identifier` are set; groups by device; calls diy-bacnet-server JSON-RPC (present-value) for each; writes `(point_id, ts, value)` to `timeseries_readings`. Throttling depends on how many points are defined and the poll interval. See Security → Throttling.

10.5 Configuration

Scraper config comes from **environment** and, when the API uses Bearer auth, from **GET /config** (the React Config page controls the interval).

- **`OFDD_BACNET_SERVER_URL`** — diy-bacnet-server base URL (e.g. http://localhost:8080). Required for RPC.
- **Scrape interval** — When **`OFDD_API_KEY`** is set (e.g. in `stack/.env`), the scraper calls GET /config and uses **`bacnet_scrape_interval_min`** from the data model (Config page). Otherwise it uses **`OFDD_BACNET_SCRAPE_INTERVAL_MIN`** env (default: 5). See Configuration → Services that read config from the API.
- **`OFDD_DB_*`** — TimescaleDB connection

11 BACnet, RDF, and BRICK

12 BACnet, RDF, and BRICK

Open-FDD treats the **building as a knowledge graph**: sites, equipment, and points (and BACnet topology) live in a single semantic model. CRUD and **POST /bacnet/point_discovery_to_graph** update this model; SPARQL queries it.

12.1 Roles

Layer	Responsibility
diy-bacnet-server	

Layer	Responsibility
	BACnet RPCs: Who-Is, point discovery (returns JSON objects per device). Open-FDD calls these.
Open-FDD	Calls point discovery, builds clean BACnet TTL from the JSON, merges into the in-memory graph (Brick from DB + BACnet). CRUD owns sites, equipment, points. SPARQL runs over the combined model; sync writes `config/data_model.ttl`.

12.2 Flow

1. **Point discovery** — Open-FDD calls **POST /bacnet/point_discovery** (gateway RPC) for a device instance; gateway returns a list of objects (object_identifier, object_name, etc.).
2. **Point discovery to graph** — **POST /bacnet/point_discovery_to_graph** (same instance, `update_graph: true`) turns that JSON into clean BACnet TTL and updates the in-memory graph; optionally writes `config/data_model.ttl`. SPARQL and **GET /data-model/ttl** see Brick + BACnet.
3. **Points in DB** — Create or edit points via CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`). Optionally **GET /data-model/export** then LLM or human adds Brick types then **PUT /data-model/import**.
4. **Scrape** — Data-model driven (points with `bacnet_device_id` / `object_identifier`).

12.3 Try it (Open-FDD Swagger)

1. Who-Is
- Open **http://localhost:8000/docs**. Find **POST /bacnet/whois_range**. Body: `{"request": {"start_instance": 1, "end_instance": 3456800}}`. Execute; note a device instance (e.g. `3456789`).

2. Point discovery to graph
- Find **POST /bacnet/point_discovery_to_graph**. Body: `{"instance": {"device_instance": 3456789}, "update_graph": true, "write_file": true}`. Execute. Open-FDD calls the gateway, gets point discovery JSON, builds BACnet TTL, and updates the in-memory graph and file.

3. Check
- **GET /data-model/ttl** or **POST /data-model/sparql** (e.g. `SELECT ?dev WHERE { ?dev a bacnet:Device }`).

12.4 Summary

- **Building as knowledge graph**: one semantic model (BRICK + BACnet), updated by CRUD and by **point_discovery_to_graph**.
- **BACnet in the graph** is produced by Open-FDD from point discovery JSON (clean TTL); SPARQL validates and queries the combined model.

13 BACnet

14 BACnet

BACnet is the **default data driver** for Open-FDD. Discovery, diy-bacnet-server, and the BACnet scraper are documented here. **Swagger (diy-bacnet-server):** http://localhost:8080/docs

---

14.1 Setup (do this before the platform scrapes data)

Recommended: data model

1. **Start the platform** (including diy-bacnet-server) — e.g. `./scripts/bootstrap.sh`.
2. **Discover devices and points** — In the Config UI (`/app/`) use the BACnet panel: Who-Is range, then Point discovery per device. Or call the API: `POST /bacnet/whois_range`, `POST /bacnet/point_discovery`.
3. **Graph and data model** — Call **POST /bacnet/point_discovery_to_graph** (device instance) to put BACnet devices and points into the in-memory graph and sync `config/data_model.ttl`. Create points in the DB via CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`) or use **GET** /

data-model/export → LLM/human tagging → **PUT /data-model/import**.

4. **Scraper runs automatically** — The bacnet-scraper container (or `tools/run_bacnet_scrape.py`) loads points from the data model and polls only those. Configuration is via the frontend and data model only; there is no CSV path.

15 Overview

16 Data model flow

Open-FDD uses a **unified graph**: one semantic model that combines Brick (sites, equipment, points), BACnet discovery RDF (from bacpypes3 in diy-bacnet-server), platform config, and—as the project evolves—other ontologies such as ASHRAE 223P. CRUD and discovery both update this graph; **all backend queries are SPARQL-driven** (rdflib Graph parse + SPARQL; no grep or text search on the TTL). Rules resolve inputs via `ofdd:mapsToRuleInput` in the TTL.

16.1 Flow

```
Sites + Equipment + Points (DB)  ← single source of truth
         │
         ▼
  Data-model export / CRUD
         │
         ▼
  Brick TTL (config/data_model.ttl)  ← Brick section reserialized
on every create/update/delete; same file can include a BACnet
discovery section (one file for SPARQL)
         │
         ▼
  FDD column_map (external_id → rule_input)
         │
         ▼
  RuleRunner
```

CRUD and Brick TTL sync: The database is the single source of truth. Every **create**, **update**, or **delete** on **sites**, **equipment**, or **points** (via API or data-model import) triggers a reserialize: the Brick TTL file (`config/data_model.ttl`, or `OFDD_BRICK_TTL_PATH`) is regenerated from

the current DB and written to disk. So the Brick model is always in sync with CRUD. Deleting a site, device (equipment), or point also cascades to dependent data (timeseries, fault results, etc.) as in a typical CRUD app; see Danger zone — CRUD deletes.

16.2 TTL structure

```
:oat_sensor a brick:Outside_Air_Temperature_Sensor ;
    rdfs:label "OAT (°F)" ;
    brick:isPointOf :ahu_7 ;
    ofdd:mapsToRuleInput "oat" .
```

- Brick classes define sensor types
- `ofdd:mapsToRuleInput` maps to FDD DataFrame columns
- `rdfs:label` for display

16.3 Data-model API

Endpoint	Description
`GET /data-model/export`	Single export route: BACnet discovery + DB points (optional `?bacnet_only=true`, `?site_id=...`). Use for AI-assisted tagging.
`PUT /data-model/import`	Import JSON: **points** (required) and optional **equipment** (feeds/fed_by). Creates/updates points; does not accept sites or equipments.
`GET /data-model/ttl`	Generate Brick TTL from DB (and in-memory BACnet graph). Optional `?save=true` to write to file.
`POST /data-model/sparql`	Run SPARQL query against the current data model.

16.4 Validation

Use SPARQL to validate:

- All rule inputs have `ofdd:mapsToRuleInput`

- Equipment types and points are consistent
- Brick schema compliance (optional)

17 Sites

18 Sites

Sites represent buildings or facilities. Equipment and points are scoped to a site.

18.1 Structure

Field	Description
`id`	UUID primary key
`name`	Display name

18.2 Scoping

All time-series and equipment queries can be filtered by `site_id`. Grafana dashboards typically scope by site.

18.3 API

- `GET /sites` — List sites
- `GET /sites/{id}` — Get one
- `POST /sites` — Create
- `PATCH /sites/{id}` — Update
- `DELETE /sites/{id}` — Delete (cascades to equipment, points, timeseries, fault results; see Danger zone)

19 Equipment

20 Equipment

Equipment represents physical devices or systems (AHUs, VAVs, heat pumps, weather stations). Points may belong to equipment.

20.1 Structure

Field	Description
`id`	UUID primary key
`site_id`	FK to sites
`name`	Display name (e.g. “AHU-7”)
`equipment_type`	Optional type string
`external_id`	Optional raw identifier from source

20.2 Hierarchy

```
Site
 └── Equipment (AHU, VAV, etc.)
      └── Points (sensors, setpoints)
```

20.3 API

- `GET /equipment` — List equipment (filter by site)
- `GET /equipment/{id}` — Get one
- `POST /equipment` — Create
- `PATCH /equipment/{id}` — Update
- `DELETE /equipment/{id}` — Delete (cascades to points and their timeseries; see Danger zone)

21 AI-assisted data modeling

22 AI-assisted Brick tagging

Open-FDD supports **AI-assisted data modeling**: use **GET /data-model/export** to dump BACnet discovery and existing points as JSON, then have an **LLM or human** add Brick types, rule inputs, equipment relationships, and which points to poll — and send the result to **PUT /data-model/import**. The platform stays the single source of truth; tagging is the step where raw BACnet objects become a curated, rule-ready dataset.

This workflow is intended for **mechanical engineers and building operators** who need the data model tagged so that FDD rules in `analyst/rules/` have the correct inputs and only the right points are logged long-term.

22.1 Workflow (export → tag → import)

1. **Discover** — Use the API: **POST /bacnet/whois_range**, then **POST /bacnet/point_discovery_to_graph** per device. The in-memory graph and `config/data_model.ttl` now contain BACnet devices and objects.

2. **Sites and equipment** — Create the building/site and equipment (AHUs, VAVs, zones) via **POST /sites** and **POST /equipment**. Note the returned `site_id` and `equipment_id` UUIDs; the import body requires real UUIDs from the API.

3. **Export** — **GET /data-model/export** (or `?bacnet_only=true` for discovery-only). Returns a single JSON array: BACnet discovery rows plus all DB points. Unimported BACnet rows have `point_id: null`, `polling: false`, and null `site_id`/`equipment_id`/`brick_type`/`rule_input`.

4. **Tag** — Send the export JSON to an LLM or edit manually. For each point set:

   - **site_id**, **external_id** (time-series key)
   - **brick_type** (e.g. `Supply_Air_Temperature_Sensor`, `Zone_Air_Temperature_Sensor`)

- **rule_input** (name FDD rules use)
- **unit** when known (e.g. `degF`, `%`, `cfm`, `0/1` for binary). Units are stored in the data model and TTL; the frontend uses them for Plots axis labels and grouping (e.g. temperatures on one axis, humidity on another). Use standard abbreviations so Plots and exports stay consistent.
- **equipment_id** (optional)
- **polling: true** for every point that must be **logged long-term** for this job (sensors/setpoints that FDD rules use); **polling: false** for points not needed. The LLM should tell the operator which points will be logged so they can confirm rules in `analyst/rules/` have the required inputs.
- For **equipment relationships** (Brick feeds/isFedBy), use the import **equipment** array: each item has `equipment_id`, optional `feeds_equipment_id` and `fed_by_equipment_id` (UUIDs from GET /equipment).

5. **Import** — **PUT /data-model/import** with body: **points** (array) and optional **equipment** (array for feeds/fed_by only). The API accepts **only** these two keys — no `sites`, `equipments`, or `relationships`. The backend creates/updates points and equipment relationships, then rebuilds the RDF and TTL.

6. **Scraping** — The BACnet scraper (data-model path) loads points where `bacnet_device_id`, `object_identifier`, and **polling** = **true**; it calls diy-bacnet-server **client_read_multiple** per device and writes to `timeseries_readings`. Grafana and FDD use the same data model.

22.2 API contract (import)

- **points** (required): Each row is a point to **create** (omit `point_id`; set `site_id`, `external_id`, `bacnet_device_id`, `object_identifier`) or **update** (set `point_id` and fields to change). **site_id** and **equipment_id** must be real UUIDs from GET /sites and GET /equipment.
- **equipment** (optional): Updates existing equipment with Brick feeds/isFedBy. Each item: `{ "equipment_id": "<uuid>", "feeds_equipment_id": "<uuid> | null", "fed_by_equipment_id": "<uuid> | null" }`.

diy-bacnet ready: The list of points to poll is **GET /data-model/export** filtered to rows where **polling** === **true**; each has `bacnet_device_id` and `object_identifier`.

22.3 LLM prompt and agent guidelines

The prompt in the README is **generic** and works for **any site**: single building, campus, or tenant. The only input that changes is the export JSON (from GET /data-model/export, optionally with `?site_id=YourSiteName`). The LLM must preserve all fields, add brick_type, rule_input, **unit** (when known), and polling, and use equipment by name and site_id from the export.

For exact schema details and import body (points + equipment only), see the Technical reference. It defines the primary task (Brick tagging), the export → tag → import flow, polling semantics, equipment feeds, and the **unit** field (e.g. degrees-fahrenheit, percent) used by the frontend and stored in the RDF/TTL.

22.4 See also

- Data model overview — Flow (DB → TTL → FDD)
- Appendix: API Reference — Data model export/import, CRUD
- BACnet overview — Discovery and data-model scrape
- Fault rules — Brick-driven rules in `analyst/rules/`

23 Points

24 Points

Points are time-series references in the Open-FDD data model. They link raw telemetry (BACnet objects, weather sensors) to equipment and Brick semantics.

24.1 Structure

Each point has:

Field	Description
`id`	UUID primary key
`site_id`	FK to sites
`equipment_id`	FK to equipment (nullable)
`external_id`	Raw identifier from source (e.g. BACnet object name); unique per site
`fdd_input`	Column name used by FDD rules (e.g. `oat`, `sat`); data-model API uses `rule_input`
`brick_type`	Optional Brick class (e.g. `Outside_Air_Temperature_Sensor`)
`unit`	Optional unit of measure
`description`	Optional
BACnet addressing	
`bacnet_device_id`	Optional. BACnet device instance (e.g. `3456789`). With `object_identifier`, used by the BACnet scraper to poll this point.
`object_identifier`	Optional. BACnet object ID (e.g. `analog-input,1`).
`object_name`	Optional. BACnet object name (often same as `external_id`).

24.2 Time-series data

Readings are stored in `timeseries_readings`:

Column	Type	Description
`point_id`	UUID	FK to points
`ts`	timestamp	Timestamp (UTC)
`value`	float	Numeric value

TimescaleDB hypertable, optimized for range scans and downsampling.

24.3 Layers and mapping

- **BACnet layer:** Points that have `bacnet_device_id` and `object_identifier` are scraped by the BACnet driver (data-model path). Add them via CRUD or after **POST /bacnet/point_discovery_to_graph** and data-model export/import. `external_id` is typically the BACnet object name.
- **Weather layer:** Points with `external_id` = `temp_f`, `rh_pct`, `dewpoint_f`, etc. come from the Open-Meteo weather fetch. They are linked to a synthetic equipment **Open-Meteo** (type **Weather_Service**) per site so they appear under "Open-Meteo" (web weather) in the **Data model** and **Points** tree. In the RDF graph that equipment is tagged with `ofdd:dataSource "open_meteo"` so you can query it via SPARQL.
- **Rule layer:** `fdd_input` / `rule_input` maps to DataFrame column names used by YAML rules.

The data-model API and Brick TTL coordinate `external_id` ↔ `rule_input` ↔ `brick_type`.

24.3.1 BACnet addressing

Points with `bacnet_device_id` and `object_identifier` set are used by the BACnet scraper (data-model path). Add them via CRUD or after point_discovery_to_graph and data-model export/import. See BACnet overview.

24.4 API

- `GET /points` — List points (filter by site or equipment)
- `GET /points/{id}` — Get one
- `POST /points` — Create
- `PATCH /points/{id}` — Update
- `DELETE /points/{id}` — Delete (cascades to timeseries_readings; see Danger zone)

24.5 Queries

```
SELECT ts, p.external_id, tr.value
FROM timeseries_readings tr
JOIN points p ON p.id = tr.point_id
```

```sql
WHERE p.external_id = 'temp_f'
  AND ts > NOW() - INTERVAL '1 day'
ORDER BY ts DESC;
```

25 SPARQL cookbook

26 SPARQL cookbook

Open-FDD keeps one **knowledge graph** (Brick + BACnet + platform config) in `config/data_model.ttl`. All queries in this cookbook run **via the REST API only** — use **POST /data-model/sparql** with a JSON body `{"query": "..."}`. Do not query the TTL file directly; the API is the single entry point so the same flow works for UIs, scripts, and tests.

26.1 How to run SPARQL

Method	Use
Swagger	http://localhost:8000/docs → **POST /data-model/sparql** → body `{"query": "SELECT ..."}` → Execute.
curl	`curl -X POST http://localhost:8000/data-model/sparql -H "Content-Type: application/json" -d '{"query":"SELECT ?s ?p ?o WHERE { ?s ?p ?o } LIMIT 10"}'`
Upload .sparql file	**POST /data-model/sparql/upload** with a `.sparql` file (e.g. from `analyst/sparql/`).

Response: JSON with a `bindings` array; each element is a map of variable names to values (e.g. `{"site_label": {"value": "DemoSite"}}`). Use the `value` field for literals and the full object for URIs.

26.2 Prefixes and graph contents

The graph uses these namespaces. Include them in your queries:

Prefix	Namespace
`brick:`	https://brickschema.org/schema/Brick#
`rdfs:`	http://www.w3.org/2000/01/rdf-schema#
`ofdd:`	http://openfdd.local/ontology#
`bacnet:`	http://data.ashrae.org/bacnet/2020#
`:` (default)	http://openfdd.local/site#

Graph contents: Brick triples (sites, equipment, points from the DB), BACnet triples (devices and objects from discovery), and **ofdd:PlatformConfig** (platform config from GET/PUT /config). All are queryable together.

26.3 Recipe 1: Platform config

Platform config is stored as RDF (same graph). Use it to verify or audit settings after PUT /config or bootstrap.

All config triples:

```
PREFIX ofdd: <http://openfdd.local/ontology#>
SELECT ?p ?v WHERE {
  ?c a ofdd:PlatformConfig .
  ?c ?p ?v .
}
```

Example bindings: `ofdd:ruleIntervalHours`, `ofdd:bacnetServerUrl`, `ofdd:bacnetEnabled`, etc. Predicates are camelCase (e.g. `ruleIntervalHours`). Use this to confirm config is in the graph after seeding or after running `tools/graph_and_crud_test.py` (steps 1c–1e).

26.4 Recipe 2: Sites and counts

All sites with labels:

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site ?site_label WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
}
```

Site and aggregate counts (equipment count, point count per site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site_label (COUNT(DISTINCT ?equipment) AS ?
equipment_count) (COUNT(?point) AS ?point_count) WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
  ?equipment brick:isPartOf ?site .
  ?point brick:isPointOf ?equipment .
}
GROUP BY ?site ?site_label
```

Use this to validate the data model after CRUD or import (e.g. in e2e tests or from `tools/graph_and_crud_test.py`; see Getting started).

26.5 Recipe 3: Equipment and points by site

Equipment labels for a given site (bind site label in the query or filter in application code):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?eq_label WHERE {
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "DemoSite" .
  ?eq rdfs:label ?eq_label .
}
```

Point labels for a site (points under equipment under site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?pt_label WHERE {
  ?pt brick:isPointOf ?eq .
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "DemoSite" .
```

```
  ?pt rdfs:label ?pt_label .
}
```

Equipment feeds (Brick `feeds` relationships for a site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?eq_label ?feeds_label WHERE {
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "DemoSite" .
  ?eq rdfs:label ?eq_label .
  ?eq brick:feeds ?other .
  ?other rdfs:label ?feeds_label .
}
```

26.6 Recipe 4: BACnet devices and objects

After **POST /bacnet/point_discovery_to_graph**, the graph contains BACnet devices and their objects.

All BACnet devices:

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?dev WHERE { ?dev a bacnet:Device }
```

Object count per device:

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?dev (COUNT(?obj) AS ?n) WHERE {
  ?dev a bacnet:Device .
  ?dev bacnet:contains ?obj .
}
GROUP BY ?dev
```

Object names for a device (e.g. device instance 3456789):

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?name WHERE {
  ?dev a bacnet:Device ;
       bacnet:device-instance 3456789 ;
       bacnet:contains ?obj .
  ?obj bacnet:object-name ?name .
}
```

Use these to confirm discovery results are in the graph and to cross-check with GET /data-model/export.

BACnet device and object addresses for points with polling = true (join Brick points to BACnet via matching label and object-name):

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?device_instance ?device_address ?object_identifier ?
point_label
WHERE {
  ?point ofdd:polling true ;
         rdfs:label ?point_label .
  ?bacnet_obj bacnet:object-name ?point_label ;
              bacnet:object-identifier ?object_identifier .
  ?device bacnet:contains ?bacnet_obj ;
          bacnet:device-instance ?device_instance ;
          bacnet:device-address ?device_address .
}
ORDER BY ?device_instance ?object_identifier
```

Returns one row per polling point that has a matching BACnet object: `device_instance`, `device_address`, `object_identifier` (e.g. `analog-input, 1`), and `point_label`. Use this for scraper config or to list "which BACnet addresses are in the data model and polled."

Why "No bindings (empty result)"? The query above joins **Brick** points to **BACnet** objects. If your graph has no BACnet triples (no discovery has been run, or you only have Brick from a TTL file), there are no `bacnet:Device`, `bacnet:contains`, or `bacnet:object-name` triples, so the join returns nothing. Use the query below when you only have Brick data.

Polling points with a Brick type (no BACnet) — use this when the graph has no BACnet data; it returns points that have `ofdd:polling true` and a Brick class (e.g. `Return_Air_Temperature_Sensor`):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?brick_class ?point_label ?unit
WHERE {
  ?point ofdd:polling true ;
         rdfs:label ?point_label ;
         a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  OPTIONAL { ?point ofdd:unit ?unit . }
```

```
}
ORDER BY ?point_label
```

Data Model page: Equipment table vs "View full data model (TTL)" — The **Equipment** table on the Data Model page comes from the **database** (REST API: sites, equipment, points). The **SPARQL** box and **"View full data model (TTL)"** use the **in-memory graph**, which is synced from the DB (Brick) plus BACnet discovery. If you see a full TTL with equipment but "No equipment configured" in the table, the DB may be empty or out of sync: the graph can be loaded from `config/data_model.ttl` on startup, but the UI list is always from the API/DB. Use the site selector in the top bar ("All sites" vs a specific site); ensure sites and equipment exist in the DB (e.g. create via the UI or import).

26.7 Recipe 5: FDD — points and rule mapping

FDD rules resolve Brick classes to DataFrame columns via the TTL. These queries help you validate that points have the right Brick types and rule inputs for your rules (e.g. sensor_bounds or expression rules).

Brick class → label → rule input (timeseries reference and FDD input name):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?brick_class ?label ?rule_input WHERE {
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  ?point rdfs:label ?label .
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?brick_class ?rule_input
LIMIT 50
```

- **`rdfs:label`** = `external_id` in the DB = timeseries key (column name in the FDD DataFrame).
- **`ofdd:mapsToRuleInput`** = `fdd_input` in the DB = name the rule uses when multiple points share a Brick class.

Brick classes used in the model (count per class):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
SELECT ?brick_class (COUNT(?point) AS ?count) WHERE {
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
}
GROUP BY ?brick_class
ORDER BY DESC(?count)
```

Use this to ensure every Brick class your rules reference (e.g. `Supply_Air_Temperature_Sensor`, `Zone_Temperature_Sensor`) has at least one point in the graph.

26.8 Recipe 6: FDD — time-series references for rules

Rules in `analyst/rules/` (e.g. `sensor_bounds.yaml`, expression rules) declare **inputs** by Brick class only. The runner uses the Brick TTL (this graph) to resolve:

- **Brick class** → points of that type → **rdfs:label** (external_id) → column in the timeseries DataFrame.

So the **timeseries reference** for FDD is the point's **rdfs:label** (which equals `external_id` in the DB and is the key in `timeseries_readings` when joined via `point_id`). To list "which columns will this rule see for site X?":

Points that map to rule inputs for a site (equipment hierarchy + label = timeseries key):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?equipment_label ?point_class ?label ?rule_input WHERE {
  ?site a brick:Site .
  ?site rdfs:label "DemoSite" .
  ?equipment brick:isPartOf ?site .
  ?equipment rdfs:label ?equipment_label .
  ?point brick:isPointOf ?equipment .
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?point_class)
  ?point rdfs:label ?label .
```

```
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?equipment_label ?point_class
```

Interpretation: For each row, **`label`** is the timeseries column name (external_id). The rule runner loads timeseries by (site, external_id) and builds the DataFrame; then Brick class (and optional rule_input) select which column is used for each rule input. So this query answers "what timeseries refs (columns) does the FDD pipeline have for this site?"

Polling points only (points that the BACnet scraper writes to; exclude non-polling):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?label ?brick_class WHERE {
  ?pt a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  ?pt rdfs:label ?label .
  ?pt ofdd:polling true .
}
ORDER BY ?brick_class ?label
```

These queries mirror the logic in `analyst/sparql/01_points_rule_mapping.sparql` and `analyst/sparql/04_brick_classes_used.sparql`; run them via the API for validation or for building a small "FDD readiness" report.

26.9 Recipe 7: Integrity and completeness

Graph check — Use **GET /data-model/check** for triple counts, sites count, BACnet device count, and orphan warnings. For custom checks, use SPARQL:

Count triples:

```
SELECT (COUNT(*) AS ?n) WHERE { ?s ?p ?o }
```

Equipment hierarchy snapshot (site → equipment type → point class → rule input):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
```

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site_label ?equipment_label ?equipment_type ?point_class ?
rule_input WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
  ?equipment brick:isPartOf ?site .
  ?equipment rdfs:label ?equipment_label .
  ?equipment ofdd:equipmentType ?equipment_type .
  ?point brick:isPointOf ?equipment .
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?point_class)
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?site_label ?equipment_label ?point_class
LIMIT 100
```

26.10 E2E test coverage (graph_and_crud_test.py)

The script **`tools/graph_and_crud_test.py`** runs against a live API and exercises CRUD plus SPARQL **only via POST /data-model/sparql**. It does not read the TTL file. Coverage that matches this cookbook:

Step	What it does
0	POST /data-model/sparql (sites query) — confirms SPARQL is CRUD-only.
1c–1e	PUT /config, GET /config, then SPARQL for ofdd:PlatformConfig (Recipe 1).
1a–1a5	BACnet: server_hello, whois_range, point_discovery_to_graph; SPARQL for bacnet:Device and object names (Recipe 4).
2–3	Sites CRUD; GET /data-model/ttl.
Later	Equipment, points, import, export; SPARQL for site labels, equipment labels, point labels, equipment feeds (Recipes 2–3).
SPARQL	_sparql_site_labels(), _sparql_point_labels_for_site(), _sparql_equipment_labels_for_site(), _sparql_equipment_feeds_for_site() — all via POST /data-model/sparql.

Run the e2e test after bootstrap or after changes to confirm config, data model, and BACnet discovery are queryable through the API. See Technical reference — Unit tests and Getting started.

26.11 Saved queries and automated tests

scripts/automated_testing/sparql/ — Cookbook queries as `.sparql` files (e.g. `01_platform_config.sparql`, `02_sites_labels.sparql`, `03_equipment_labels_testbench.sparql`, `05_brick_rule_mapping.sparql`, `07_count_triples.sparql`). Use them with **POST /data-model/sparql** (paste contents) or **POST /data-model/sparql/upload** (upload file).

SPARQL CRUD + frontend parity test — From a test bench (e.g. Windows WSL), run:

```
cd /path/to/open-fdd
pip install -r scripts/automated_testing/requirements-e2e.txt
python scripts/automated_testing/sparql_crud_and_frontend_test.py
--api-url http://OPENFDD_SERVER:8000
```

With **–frontend-url** and **–frontend-parity**, the script runs each query via the API and again via the Data Model page (SPARQL textarea → Run SPARQL), then asserts the same bindings. Use **–headed** to watch the browser.

```
python scripts/automated_testing/sparql_crud_and_frontend_test.py
--api-url http://192.168.204.16:8000 --frontend-url http://
192.168.204.16 --frontend-parity --headed
```

analyst/sparql/ — Additional `.sparql` files (e.g. `01_points_rule_mapping.sparql`, `05_site_and_counts.sparql`) for FDD-oriented validation. Run via **POST /data-model/sparql/upload** or paste into Swagger / Data Model page.

26.12 Related docs

Topic	Page
Data model flow	Overview
AI-assisted tagging	AI-assisted tagging
Rule inputs and Brick	Expression rule cookbook · Fault rules overview

Topic	Page
Grafana (SQL, not SPARQL)	Grafana SQL cookbook
Dev reference	Technical reference

27 Data modeling

28 Data modeling

The Open-FDD data model is built around **sites**, **equipment**, and **points**: the same entities you create, read, update, and delete via the API (**CRUD** = Create, Read, Update, Delete). The REST API (Swagger at `/docs`) and the data-model endpoints (`/data-model/export`, `/data-model/import`, `/data-model/ttl`) are the main way to manage this data. The database is the single source of truth; the Brick TTL file is regenerated on every change so FDD rules and Grafana stay in sync.

Concepts (entities):

- **Sites** — Buildings or facilities. All equipment and points are scoped to a site.
- **Equipment** — Physical devices (AHUs, VAVs, heat pumps). Belong to a site; have points.
- **Points** — Time-series references (sensors, setpoints). Link to equipment and site; store `external_id`, optional `brick_type` and `rule_input` for FDD.
- **SPARQL cookbook** — Run SPARQL via POST /data-model/sparql only: config, data model, BACnet, FDD rule mapping, time-series references. Copy-paste queries for validation and UIs.
- **AI-assisted data modeling** — Export → LLM or human tagging → import (Brick types, rule_input, polling, equipment feeds). See also the Technical reference for the full LLM prompt and schema.

Framework: CRUD is provided by the FastAPI REST API. The data-model API adds bulk export/import and Brick TTL generation; see Overview for the flow (DB → TTL → FDD column_map) and Appendix: API Reference for endpoints.

29 Overview

30 Fault rules

Fault rules are YAML-defined checks run against time-series DataFrames. Each rule produces a boolean fault flag. **Open-FDD is 100% Brick-model driven:** rule inputs refer only to Brick classes; column mapping comes from the Brick TTL via SPARQL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

30.1 Where to place your rules

One place only: put your project rules in **`analyst/rules/`** (`.yaml` files). The FDD loop loads from this directory every run. Copy or adapt examples from `open_fdd/rules/` (sensor bounds, flatline, weather, AHU). To write new rules, see the Expression Rule Cookbook and online docs.

Config: `rules_dir: "analyst/rules"` (or `OFDD_RULES_DIR`). If that path doesn't exist, the loop falls back to `open_fdd/rules`.

30.2 Hot reload (edit → run → view in Grafana)

Open-FDD is **AFDD** (Automated Fault Detection and Diagnostics). The project **supports hot reloading of YAML rule files** so the AFDD maintainer can make tweaks and adjustments for fault tuning without restarting the platform. Rule YAML files contain the fault logic and **params** (thresholds, tolerances, windows); the FDD loop loads from `rules_dir` every run, so the rule runner does **not** need a restart. The `rules_dir` path is **RDF-driven** (platform config: GET `/config`).

1. **Edit** a rule in `analyst/rules/*.yaml` (or add a new one). Change `params` (e.g. `low`, `high`, `tolerance`, `rolling_window`) to tune fault sensitivity.
2. **Run** FDD: wait for the next scheduled run (per `rule_interval_hours` and `lookback_days` in platform config), or trigger now with `touch`

`config/.run_fdd_now` or `POST /run-fdd` (see Appendix: API Reference).

3. **View** fault results in Grafana (build a Fault Results dashboard from the Grafana SQL cookbook). Every run reloads all rules from disk, so the next run uses the latest YAML and params — hot reload style.

30.3 Rule types

Type	Purpose
`bounds`	Value outside `[low, high]`
`flatline`	Rolling spread < tolerance (stuck sensor)
`hunting`	Excessive state changes (PID hunting)
`expression`	Custom pandas/numpy expression
`oa_fraction`	OA fraction vs design airflow error
`erv_efficiency`	ERV effectiveness out of range

30.4 YAML structure

Inputs use **Brick class names** only; the Brick TTL (SPARQL) resolves each to a DataFrame column via external timeseries references. No `column` in YAML.

```yaml
name: sensor_bounds
type: bounds
flag: bad_sensor
inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
params:
  low: 40
  high: 90
  rolling_window: 6   # optional: require N consecutive True samples before flagging
```

Per-rule rolling window: Set `params.rolling_window` (e.g. `6`) in a rule to require that many consecutive True samples before the fault is flagged. Omit for "flag on any True." See `sensor_flatline.yaml`, `weather_temp_stuck.yaml`.

30.5 Input resolution (100% Brick)

- **Brick TTL:** Each rule input declares a Brick class (e.g. `brick:Supply_Air_Temperature_Sensor`). The engine resolves Brick points via SPARQL to their external timeseries reference (`ref:TimeseriesReference` / `ref:hasTimeseriesId`), yielding the DataFrame column name. See Brick timeseries storage.
- **Column map:** Built from the Brick model (e.g. `rdfs:label` or external_id) → column names; no `column` in rule YAML.

30.6 Running rules

- **Platform:** FDD loop loads rules from `rules_dir` (default `analyst/rules`) each run; edit YAML and trigger a run to see changes in Grafana.
- **Standalone:** `RuleRunner(rules_path=...)` or `RuleRunner(rules=[...])`; call `run(df, ...)`.

30.7 Cookbook

See Expression Rule Cookbook for AHU, chiller, weather, and advanced recipes.

31 Expression Rule Cookbook

32 Expression Rule Cookbook

A reference for building fault detection rules in open-fdd. Rules use **YAML** with **expression** type: when the expression evaluates to **True**, a fault is flagged. open-fdd injects **NumPy** as `np` into expression evaluation, so you can use `np.maximum`, `np.abs`, `np.sqrt`, etc. for vectorized math. **Tuning:** Change `params` in YAML and trigger an FDD run (or wait for the schedule); the platform hot-reloads rules each run, so no restart is needed. See Fault rules overview and Configuration (`rules_dir` is RDF-driven via GET `/config`).

32.1 100% Brick-model driven (no column in rules)

Rule inputs use **Brick class names** only (e.g. `Supply_Air_Temperature_Sensor`). The **Brick TTL** is the source of truth: SPARQL resolves each Brick point to its external timeseries reference (`ref:TimeseriesReference` / `ref:hasTimeseriesId`), which yields the DataFrame column. Open-FDD embraces Brick timeseries storage at the heart of FDD — **rules never reference `column`**; the data model provides the mapping. See the SPARQL cookbook (Recipe 5) and `analyst/sparql/01_points_rule_mapping.sparql`.

- **Platform (DB):** Points have `brick_type` and `external_id`; TTL is built from DB. PATCH points or use data-model import to set `brick_type` (e.g. `Supply_Air_Temperature_Sensor` for SA-T).
- **Disambiguation:** When multiple points share a Brick class, use `ofdd:mapsToRuleInput` in TTL; the runner resolves `BrickClass|rule_input`.

32.2 How to define expressions

1. **Inputs** — Declare Brick classes only (`brick:Supply_Air_Temperature_Sensor`). The runner resolves these via the Brick TTL (SPARQL) and Brick external timeseries references to DataFrame columns. Do not add `column` in YAML.
2. **Params** — Thresholds and constants go in `params`. Reference by name (e.g. `err_thresh`, `vfd_max`).
3. **Expression** — Must evaluate to a boolean Series (True = fault). Use `&` (AND), `|` (OR), `~` (NOT). Use `.diff()`, `.rolling()`, `.notna()` for time-series logic.

Minimal example (TTL-driven; no column):

```yaml
name: high_temp_check
type: expression
flag: high_temp_flag

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor

params:
  max_temp: 90.0
```

```
        expression: |
          Supply_Air_Temperature_Sensor > max_temp
```

The Brick TTL (and SPARQL) maps each Brick class to the actual column via external timeseries references. Set `points.brick_type` in the DB and sync TTL so the engine can resolve columns from the Brick model.

32.3 AHU rules (reference-style)

The following rules follow common industry practice for air-handling fault detection. Rules A through M are adapted from ASHRAE Guideline 36 (GL36) AFDD guidance. Thresholds and logic are tunable; adjust params for your site.

32.3.1 Rule A — Duct static below setpoint at full fan speed

Static pressure under setpoint while supply fan runs near maximum. May indicate duct leakage, undersized fan, or terminal damper issues. *Adapted from GL36 AFDD guidance.*

```
        name: duct_static_low_at_full_speed
        description: Static pressure below setpoint when fan at full speed
(GL36-inspired)
        type: expression
        flag: rule_a_flag
        equipment_type: [VAV_AHU]

        inputs:
          Supply_Air_Static_Pressure_Sensor:
            brick: Supply_Air_Static_Pressure_Sensor
          Supply_Air_Static_Pressure_Setpoint:
            brick: Supply_Air_Static_Pressure_Setpoint
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          sp_margin: 0.12
          drv_hi_frac: 0.93
          drv_near_hi: 0.06

        expression: |
          (Supply_Air_Static_Pressure_Sensor <
Supply_Air_Static_Pressure_Setpoint - sp_margin) &
(Supply_Fan_Speed_Command >= drv_hi_frac - drv_near_hi)
```

32.3.2 Rule B — Blended air temp below expected band

Blended air temp should lie between outdoor and return. If below both (minus tolerance), suspect sensor or mixing fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_below_band
        description: Blended air temp below expected range (OAT/RAT)
(GL36-inspired)
        type: expression
        flag: rule_b_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15

        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol <
np.minimum(Return_Air_Temperature_Sensor - rat_tol,
Outside_Air_Temperature_Sensor - oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

32.3.3 Rule C — Blended air temp above expected band

Blended air temp above the higher of OAT and RAT (plus tolerance) indicates mixing or sensor fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_above_band
        description: Blended air temp above expected range (OAT/RAT)
(GL36-inspired)
        type: expression
        flag: rule_c_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
```

```
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15

        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol >
np.maximum(Return_Air_Temperature_Sensor + rat_tol,
Outside_Air_Temperature_Sensor + oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

Hunting/oscillation — see [Hunting Rule]({{ "hunting_rule" | relative_url }}).

32.3.4 Rule D — Discharge air cold when heating commanded

Discharge air temp below blended air when heating valve is open. Indicates heating coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_cold_when_heating
        description: Discharge air below blended air when heating active
(GL36-inspired)
        type: expression
        flag: rule_d_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          sat_tol: 1.15
          fan_delta_t: 0.55

        expression: |
```

```
          (Supply_Air_Temperature_Sensor + sat_tol <=
Mixed_Air_Temperature_Sensor - blend_tol + fan_delta_t) &
(Valve_Command > 0.01) & (Supply_Fan_Speed_Command > 0.01)
```

OA fraction — see [OA Fraction Rule]({{ "oa_fraction_rule" | relative_url }}).

32.3.5 Rule E — SAT too low with full heating

Heating valve fully open but SAT remains below setpoint. Indicates undersized coil or valve failure. *Adapted from GL36 AFDD guidance.*

```yaml
        name: sat_too_low_full_heating
        description: SAT below setpoint with heating valve fully open
(GL36-inspired)
        type: expression
        flag: rule_e_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          supply_err_thres: 1.0

        expression: |
          (Supply_Air_Temperature_Sensor < Supply_Air_Temperature_Setpoint
- supply_err_thres) & (Valve_Command > 0.9) &
(Supply_Fan_Speed_Command > 0)
```

32.3.6 Rule F — SAT/MAT mismatch in economizer mode

In economizer mode (min mechanical cooling), SAT should approximate MAT. Large deviation suggests coil bypass or sensor error. *Adapted from GL36 AFDD guidance.*

```yaml
        name: discharge_blend_mismatch_econ
        description: Discharge and blended air diverge in economizer mode
(GL36-inspired)
        type: expression
        flag: rule_f_flag
        equipment_type: [AHU, VAV_AHU]
```

```
        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (np.abs(Supply_Air_Temperature_Sensor - fan_delta_t -
Mixed_Air_Temperature_Sensor) > np.sqrt(sat_tol**2 +
blend_tol**2)) & (Damper_Position_Command > econ_min_open) &
(Valve_Command < 0.1)
```

32.3.7 Rule G — Ambient too warm for free cooling

Outside air temperature exceeds SAT setpoint while economizer is active and mechanical cooling is off. Economizer should not be providing "free" cooling under these conditions. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_warm_free_cool
        description: Outdoor air above setpoint in free cooling mode
(GL36-inspired)
        type: expression
        flag: rule_g_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          oat_tol: 1.15
          fan_delta_t: 0.55
```

```
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Outside_Air_Temperature_Sensor - oat_tol >
Supply_Air_Temperature_Setpoint - fan_delta_t + sat_tol) &
(Damper_Position_Command > econ_min_open) & (Valve_Command < 0.1)
```

32.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)

When both economizer and mechanical cooling are active, MAT should approach OAT. Large deviation suggests inadequate mixing or damper fault. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_blend_mismatch_econ_mech
        description: Outdoor and blended air diverge in econ+mech cooling
(GL36-inspired)
        type: expression
        flag: rule_h_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Valve_Command > 0.01) & (Damper_Position_Command >
0.9)
```

32.3.9 Rule I — Ambient vs blended mismatch (econ-only)

In economizer-only mode, MAT should match OAT. Deviation indicates damper or mixing fault. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_blend_mismatch_econ
        description: Outdoor and blended air diverge in economizer-only
```

```
mode (GL36-inspired)
        type: expression
        flag: rule_i_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Damper_Position_Command > 0.9)
```

32.3.10 Rule J — Discharge above blended in cooling

SAT exceeds MAT when cooling (econ+mech or mech-only) is active. Indicates underperforming cooling coil or valve. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_above_blend_cooling
        description: Discharge air above blended air in cooling modes
(GL36-inspired)
        type: expression
        flag: rule_j_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12
```

```
        expression: |
          (Supply_Air_Temperature_Sensor > Mixed_Air_Temperature_Sensor +
np.sqrt(sat_tol**2 + blend_tol**2) + fan_delta_t) &
(((Damper_Position_Command > 0.9) & (Valve_Command > 0)) |
((Damper_Position_Command <= econ_min_open) & (Valve_Command >
0.9)))
```

32.3.11 Rule K — Discharge above setpoint in full cooling

SAT above setpoint with cooling at full capacity. Suggests undersized coil or plant limits. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_above_sp_full_cool
        description: Discharge air above setpoint in full cooling mode
(GL36-inspired)
        type: expression
        flag: rule_k_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Supply_Air_Temperature_Sensor > Supply_Air_Temperature_Setpoint
+ sat_tol) & (((Damper_Position_Command > 0.9) & (Valve_Command >
0.9)) | ((Damper_Position_Command <= econ_min_open) &
(Valve_Command > 0.9)))
```

32.3.12 Rule L — Cooling coil delta-T when inactive

Temperature drop across cooling coil when it should be off. Indicates leaking CHW valve or coil bypass. *Adapted from GL36 AFDD guidance.*

```
        name: clg_coil_drop_when_off
        description: Temperature drop across cooling coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_l_flag
        equipment_type: [AHU, VAV_AHU]
```

```
        inputs:
          Cooling_Coil_Entering_Air_Temperature_Sensor:
            brick: Cooling_Coil_Entering_Air_Temperature_Sensor
          Cooling_Coil_Leaving_Air_Temperature_Sensor:
            brick: Cooling_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          enter_tol: 1.15
          leave_tol: 1.15
          econ_min_open: 0.12

        expression: |
          ((Cooling_Coil_Entering_Air_Temperature_Sensor -
Cooling_Coil_Leaving_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2)) & (((Heating_Valve_Command >
0) & (Cooling_Valve_Command == 0) & (Damper_Position_Command <=
econ_min_open)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command == 0) & (Damper_Position_Command >
econ_min_open)))
```

32.3.13 Rule M — Heating coil delta-T when inactive

Temperature rise across heating coil when it should be off. Indicates leaking HW valve. *Adapted from GL36 AFDD guidance.*

```
        name: htg_coil_rise_when_off
        description: Temperature rise across heating coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_m_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Heating_Coil_Entering_Air_Temperature_Sensor:
            brick: Heating_Coil_Entering_Air_Temperature_Sensor
          Heating_Coil_Leaving_Air_Temperature_Sensor:
            brick: Heating_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command
```

```
        params:
          enter_tol: 1.15
          leave_tol: 1.15
          fan_delta_t: 0.55
          econ_min_open: 0.12

        expression: |
          ((Heating_Coil_Leaving_Air_Temperature_Sensor -
Heating_Coil_Entering_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2) + fan_delta_t) &
(((Heating_Valve_Command == 0) & (Cooling_Valve_Command == 0) &
(Damper_Position_Command > econ_min_open)) |
((Heating_Valve_Command == 0) & (Cooling_Valve_Command > 0) &
(Damper_Position_Command > 0.9)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command > 0) & (Damper_Position_Command <=
econ_min_open)))
```

Heat exchanger effectiveness — see [ERV/Heat Exchanger Rule] ({{ "erv_efficiency_rule" | relative_url }}).

32.4 Central plant

32.4.1 Differential pressure at max pump speed

Variable-speed pump cannot meet differential pressure setpoint at full speed. Indicates piping issues, undersized pump, or blocked strainers.

```
        name: dp_below_sp_pump_max
        description: Differential pressure below setpoint with pump at
full speed
        type: expression
        flag: dp_pump_flag

        inputs:
          Differential_Pressure_Sensor:
            brick: Differential_Pressure_Sensor
          Differential_Pressure_Setpoint:
            brick: Differential_Pressure_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          dp_margin: 2.2
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
```

```
          (Differential_Pressure_Sensor < Differential_Pressure_Setpoint -
dp_margin) & (Pump_Speed_Command >= pmp_hi_frac - pmp_near_hi)
```

32.4.2 Plant flow high at max pump

Flow unusually high with pump at high speed. Suggests short circuit or flow meter error.

```yaml
        name: flow_high_pump_max
        description: Water flow unusually high with pump at full speed
        type: expression
        flag: flow_high_flag

        inputs:
          Water_Flow_Sensor:
            brick: Water_Flow_Sensor
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          flow_hi_limit: 1100.0
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
          (Water_Flow_Sensor > flow_hi_limit) & (Pump_Speed_Command >=
pmp_hi_frac - pmp_near_hi)
```

32.4.3 Plant supply temp outside deadband

CHW supply temperature outside deadband while pump runs. Requires coil entering/leaving or plant supply temp sensors.

```yaml
        name: plant_supply_temp_deadband
        description: Supply water temp outside deadband during pump
operation
        type: expression
        flag: chw_temp_fault

        inputs:
          Chilled_Water_Supply_Temperature_Sensor:
            brick: Chilled_Water_Supply_Temperature_Sensor
          Chilled_Water_Supply_Temperature_Setpoint:
            brick: Chilled_Water_Supply_Temperature_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          sp_band: 2.2
```

```
        expression: |
          (Pump_Speed_Command > 0.01) &
((Chilled_Water_Supply_Temperature_Sensor <
Chilled_Water_Supply_Temperature_Setpoint - sp_band) |
(Chilled_Water_Supply_Temperature_Sensor >
Chilled_Water_Supply_Temperature_Setpoint + sp_band))
```

32.4.4 Chiller runtime over daily limit

Chiller running beyond a daily threshold (e.g. 23 hours). Often indicates over-cooling or schedules that bypass lockout. Use a rolling window sized for your data interval (e.g. 5-min data → 276 samples ≈ 23 h).

```
        name: chiller_excessive_runtime
        description: Chiller runtime exceeds daily threshold (rolling
window)
        type: expression
        flag: chiller_runtime_fault

        inputs:
          Chiller_Status:
            brick: Chiller_Status

        params:
          # For 5-min data: 23 hours ≈ 276 samples; max_runtime = count of
"on" samples in window
          rolling_samples: 276
          max_runtime_samples: 264

        expression: |
          Chiller_Status.rolling(window=rolling_samples).sum() >
max_runtime_samples
```

Note: Adjust `rolling_samples` and `max_runtime_samples` for your data interval. For 5-min data, 264 samples ≈ 22 hours.

32.5 Heat pumps

32.5.1 Discharge cold when heating

Flags when discharge air temperature is below a minimum (e.g. 80°F) while the supply fan is running. Indicates the heat pump is not heating effectively—possible issues with compressor, refrigerant, or reversing

valve. Tunable via `min_discharge_temp`. Logic: if zone temp < 69°F (heating mode), the discharge should be warm; a cold discharge with the fan on and a cold zone indicates the heat pump is failing to heat.

```
        name: hp_discharge_cold_when_heating
        description: Discharge air below minimum when fan on and zone is
cold (heating mode)
        type: expression
        flag: hp_discharge_cold_flag
        equipment_type: [Heat_Pump]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Zone_Temperature_Sensor:
            brick: Zone_Temperature_Sensor
          Supply_Fan_Status:
            brick: Supply_Fan_Status

        params:
          min_discharge_temp: 85
          zone_cold_threshold: 69.0
          fan_on_threshold: 0.01

        expression: |
          (Supply_Fan_Status > fan_on_threshold) &
(Zone_Temperature_Sensor < zone_cold_threshold) &
(Supply_Air_Temperature_Sensor < min_discharge_temp)
```

32.6 VAV zones

32.6.1 Excessive heating during warm weather

Reheat valve open when outdoor air is warm. Suggests over-cooling or setpoint issues.

```
        name: zone_reheat_warm_ambient
        description: Heating valve open when OAT is high
        type: expression
        flag: excessive_heating_flag
        equipment_type: [VAV]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Reheat_Valve_Command:
            brick: Valve_Command
```

```
        params:
          t_amb_cutoff: 78.0
          reheat_open_min: 0.52

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_cutoff) &
(Reheat_Valve_Command > reheat_open_min)
```

32.6.2 Damper or valve at full open

Damper or reheat valve consistently at full open. Indicates override, undersized equipment, or control fault.

```
        name: zone_damper_valve_full_open
        description: Damper or valve at full open for extended period
        type: expression
        flag: damper_100_flag
        equipment_type: [VAV]

        inputs:
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          full_open_pct: 97.5
          roll_samples: 105

        expression: |
          (Damper_Position_Command > full_open_pct) &
(Damper_Position_Command.rolling(roll_samples).min() >
full_open_pct)
```

32.6.3 Zone and IAQ bounds

For CO2 and zone temperature out-of-range checks, use the [Bounds Rule]({{ “bounds_rule” | relative_url }}) — `co2_bounds` and `zone_temp_bounds` examples.

32.7 Opportunistic rules (economizer & ventilation)

32.7.1 Economizing when outdoor conditions are unfavorable

Economizer active when outdoor air is too warm or humid. Use OAT (or enthalpy if available) vs. threshold.

```
        name: econ_active_warm_ambient
        description: OA damper open when outdoor conditions do not favor
economizing
        type: expression
        flag: econ_when_shouldnt_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_econ_min: 0.42

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_econ_cutoff) &
(Damper_Position_Command > dpr_econ_min)
```

32.7.2 Mechanical cooling when econ could suffice

Cooling valve open when outdoor air is cool enough for economizer. Opportunity to reduce mechanical cooling.

```
        name: mech_cool_when_econ_available
        description: Mechanical cooling active when economizing could
suffice
        type: expression
        flag: cooling_when_econ_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
```

```
            brick: Valve_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_not_econ_max: 0.32

        expression: |
          (Outside_Air_Temperature_Sensor < t_amb_econ_cutoff) &
(Damper_Position_Command < dpr_not_econ_max) & (Valve_Command >
0.01)
```

32.7.3 Low ventilation (estimated OA fraction)

For units without airflow meters, estimate OA fraction from OAT, MAT, RAT. Flag when below minimum design OA.

```
        name: low_oa_fraction_estimated
        description: Estimated OA fraction below minimum (OAT, MAT, RAT
method)
        type: expression
        flag: low_vent_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          oa_min_pct: 21.0
          t_rat_oat_min_gap: 2.2

        expression: |
          (Supply_Fan_Speed_Command > 0.01) &
(np.abs(Return_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > t_rat_oat_min_gap) &
(((Mixed_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) /
(Outside_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) *
100) < oa_min_pct)
```

Note: Guard against division-by-zero when outdoor and return temps are close.

32.7.4 Preheat over-conditioning

Preheat coil leaving temp higher than needed (e.g. above OAT when OAT > SAT SP, or above SAT SP when OAT < SAT SP). Indicates wasted heating energy.

```yaml
name: preheat_excess_temp
description: Preheat coil leaving temp above required level
type: expression
flag: preheat_waste_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Preheat_Coil_Leaving_Air_Temperature_Sensor:
    brick: Preheat_Coil_Leaving_Air_Temperature_Sensor
  Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Valve_Command:
    brick: Valve_Command

params:
  excess_tol: 2.2

expression: |
  (Valve_Command > 0.01) & (((Outside_Air_Temperature_Sensor >
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor > excess_tol)) |
((Outside_Air_Temperature_Sensor <
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Supply_Air_Temperature_Setpoint > excess_tol)))
```

32.7.5 Blended air damper deviation

Expected MAT (from OAT, RAT, damper positions) differs from measured MAT. Indicates damper leakage or faulty mixing.

Requires airflow or damper position data; logic is more involved. A simplified version compares MAT to a weighted blend of OAT and RAT when dampers suggest significant OA.

32.8 Weather station

weather_temp_stuck (flatline) — see [Flatline Rule]({{ “flatline_rule” | relative_url }}).

32.8.1 Unrealistic temperature spike

Temperature change between consecutive readings exceeds physical limit.

```yaml
name: weather_temp_spike
description: Unrealistic temperature change between readings
type: expression
flag: fault_temp_spike

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor

params:
  spike_limit: 16.0

expression: |
  Outside_Air_Temperature_Sensor.diff().abs() > spike_limit
```

32.8.2 RH bounds

For relative humidity out-of-range, use the [Bounds Rule] ({{ “bounds_rule” | relative_url }}) — `rh_bounds` example.

32.8.3 Wind gust vs sustained

```yaml
name: weather_gust_lt_wind
description: Wind gust reported lower than sustained wind
type: expression
flag: fault_gust_lt_wind

inputs:
  Wind_Gust_Speed_Sensor:
    brick: Wind_Gust_Speed_Sensor
  Wind_Speed_Sensor:
    brick: Wind_Speed_Sensor

expression: |
  Wind_Gust_Speed_Sensor.notna() & Wind_Speed_Sensor.notna() &
(Wind_Gust_Speed_Sensor < Wind_Speed_Sensor)
```

32.9 Sensor validation (bounds & flatline)

Use the [Bounds Rule]({{ “bounds_rule” | relative_url }}) and [Flatline Rule]({{ “flatline_rule” | relative_url }}) for generic sensor checks. Typical bounds:

Sensor type	Min	Max
Zone temp	40	100
Supply air temp	40	150
Air pressure (inH2O)	-5	10
RH	0	100
Chilled water temp	35	100
Hot water temp	50	212
Condenser water	50	110
CO2 (ppm)	400	2000

Next: [Flat Line Sensor Tutorial]({{ “flat_line_sensor_tutorial” | relative_url }})

33 Fault rules for HVAC

34 Fault rules for HVAC

FDD fault rule types, YAML format, and expression cookbook for HVAC and building systems. Rules are **100% Brick-model driven**: inputs reference Brick classes only; column mapping comes from the Brick TTL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

35 Cloud export example

36 Cloud export example

Open-FDD runs **inside the building, behind the firewall**. It does not push data to the cloud itself. Cloud-based FDD providers, MSI (Monitoring, Savings, and Intelligence) vendors, and commissioning or IoT contractors run their **own** export process: a small script or edge gateway on the building or OT network that **pulls** from the Open-FDD API over the LAN and then sends that data to their cloud or analytics platform. See Behind the firewall; cloud export is vendor-led on the docs home.

This page describes the **cloud export example** in the repo: a minimal Python script that demonstrates how vendor X, Y, or Z could use the Open-FDD API to get fault and timeseries data and use it as a starting point for their cloud pipeline.

36.1 What the example does

The script `examples/cloud_export.py` calls standard Open-FDD download and analytics endpoints and prints the results. In a real integration you would replace the print steps with your own logic: POST to your REST API, write to S3, push to Azure IoT Hub or AWS IoT, send to SkySpark or another analytics platform, etc.

Step	Endpoint	Purpose
1	`GET /download/faults?format=json`	Fault results for programmatic ingestion (JSON)
2	`GET /download/faults?format=csv`	Fault CSV (Excel-friendly)
3	`GET /analytics/motor-runtime`	Motor runtime (data-model driven)
4	`GET /download/csv`	Timeseries wide-format CSV
5		Fault counts by fault_id

Step	Endpoint	Purpose
	`GET /analytics/fault-summary`	

All requests use date range and optional `site_id`; the API docs at `/docs` list full parameters.

36.2 Run the example

Prerequisites: Open-FDD API running (e.g. `http://localhost:8000`), `pip install httpx`.

```
# From repo root
python examples/cloud_export.py
python examples/cloud_export.py --site default --days 14
API_BASE=http://your-openfdd-host:8000 python examples/cloud_export.py
```

Docker: See `examples/cloud_export/README.md` for building and running the example in a container (e.g. on a vendor edge device that reaches the Open-FDD API over the LAN).

36.3 Use it as your starting point

1. **Clone or copy** `examples/cloud_export.py` into your integration codebase.
2. **Keep** the HTTP calls to `/download/faults`, `/download/csv`, and the analytics endpoints; adjust `API_BASE` (or env) to point at the customer's Open-FDD instance.
3. **Replace** the `print()` / local handling with your cloud send: REST POST, message queue, blob storage, or IoT hub. Add auth (e.g. API key or TLS client cert) if you expose the puller to a wider network.
4. **Schedule** the script (cron, systemd timer, or container restart policy) so you poll the Open-FDD API at the interval that fits your product (e.g. hourly or daily fault sync).

Open-FDD does not initiate outbound cloud connections or manage your data transmission; the example shows how your process can pull data from Open-FDD and then send it to your cloud for deeper insights.

37 Concepts

38 Concepts

Conceptual guides and worked examples for how Open-FDD fits into larger workflows.

Page	Description
Cloud export example	How a cloud vendor or MSI can pull fault and timeseries data from the Open-FDD API and send it to their cloud for deeper insights. Uses the `examples/cloud_export` script as a starting point.

See also Behind the firewall; cloud export is vendor-led on the docs home.

39 How-to Guides

40 How-to Guides

- Grafana dashboards (optional) — Optional Grafana; React frontend provides equivalent timeseries and fault views. Datasource and dashboard JSON.
- Grafana SQL cookbook — SQL recipes for BACnet, faults, weather, system resources (variables, panels, sparklines).

41 Grafana SQL cookbook

42 Grafana SQL cookbook —

42.1 BACnet

- Dashboard Variables (Required for Dropdowns)

Before creating the panels below, you must set up the dashboard variables so the drop-down menus work.

In Grafana, go to **Dashboard Settings (gear icon)** > **Variables** > **Add variable**. Set the **Type** to `Query`, select your TimescaleDB datasource, and use the following queries:

42.1.1 1. Site Variable

- **Name:** `site`
- **Query:**

```
SELECT s.name AS __text, s.id::text AS __value
FROM sites s
ORDER BY s.name;
```

42.1.2 2. Device Variable

- **Name:** `device`
- **Query:**

```
SELECT DISTINCT p.bacnet_device_id::text AS __text,
p.bacnet_device_id::text AS __value
FROM points p
WHERE p.site_id::text = '$site'
  AND p.bacnet_device_id IS NOT NULL
ORDER BY 1;
```

42.1.3 3. Point Variable

- **Name:** `point`
- **Query:**

```sql
SELECT p.external_id AS __text, p.external_id AS __value
FROM points p
WHERE p.site_id::text = '$site'
  AND p.bacnet_device_id::text = '$device'
ORDER BY 1;
```

42.1.4 BACnet only data

Insert Snip here

- This dashboard uses `point`, `device`, and `site` variables.

```json
{
  "id": 1,
  "type": "timeseries",
  "title": "BACnet",
  "gridPos": {
    "x": 0,
    "y": 0,
    "h": 16,
    "w": 24
  },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "drawStyle": "line",
        "lineInterpolation": "linear",
        "barAlignment": 0,
        "barWidthFactor": 0.6,
        "lineWidth": 1,
        "fillOpacity": 0,
        "gradientMode": "none",
        "spanNulls": false,
        "insertNulls": false,
        "showPoints": "auto",
        "showValues": false,
        "pointSize": 5,
        "stacking": {
          "mode": "none",
          "group": "A"
        },
        "axisPlacement": "auto",
        "axisLabel": "",
        "axisColorMode": "text",
        "axisBorderShow": false,
        "scaleDistribution": {
          "type": "linear"
        },
        "axisCenteredZero": false,
        "hideFrom": {
```

```
                "tooltip": false,
                "viz": false,
                "legend": false
              },
              "thresholdsStyle": {
                "mode": "off"
              }
            },
            "color": {
              "mode": "palette-classic"
            },
            "mappings": [],
            "thresholds": {
              "mode": "absolute",
              "steps": [
                {
                  "color": "green",
                  "value": null
                },
                {
                  "color": "red",
                  "value": 80
                }
              ]
            }
          },
          "overrides": []
        },
        "pluginVersion": "12.4.0",
        "targets": [
          {
            "editorMode": "code",
            "format": "time_series",
            "rawQuery": true,
            "rawSql": "SELECT\n
time_bucket(\n    make_interval(secs => ($__interval_ms /
1000)::int),\n    tr.ts\n  ) AS \"time\",\n  avg(tr.value) AS
\"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.bacnet_device_id::text IN ($
{device:sqlstring})\n  AND p.external_id IN (${point:sqlstring})
\nGROUP BY 1, 3\nORDER BY 1, 3;",
            "refId": "A",
            "sql": {
              "columns": [
                {
                  "parameters": [],
                  "type": "function"
                }
              ],
              "groupBy": [
```

```
            {
              "property": {
                "type": "string"
              },
              "type": "groupBy"
            }
          ],
          "limit": 50
        }
      }
    ],
    "datasource": {
      "uid": "openfdd_timescale"
    },
    "options": {
      "tooltip": {
        "mode": "single",
        "sort": "none",
        "hideZeros": false
      },
      "legend": {
        "showLegend": true,
        "displayMode": "list",
        "placement": "bottom",
        "calcs": []
      }
    }
  }
```

42.2 BACnet Plus Fault Data

```
  {
    "id": null,
    "type": "timeseries",
    "title": "BACnet Telemetry w/ Fault Overlays",
    "gridPos": {
      "x": 0,
      "y": 16,
      "h": 12,
      "w": 24
    },
    "fieldConfig": {
      "defaults": {
        "custom": {
          "drawStyle": "line",
          "lineInterpolation": "linear",
          "barAlignment": 0,
          "barWidthFactor": 0.6,
          "lineWidth": 2,
          "fillOpacity": 10,
```

```
            "gradientMode": "none",
            "spanNulls": false,
            "insertNulls": false,
            "showPoints": "never",
            "showValues": false,
            "pointSize": 5,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": false,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "value": null,
                "color": "green"
              },
              {
                "value": 80,
                "color": "red"
              }
            ]
          }
        },
        "overrides": [
          {
            "matcher": {
              "id": "byFrameRefID",
              "options": "B"
            },
            "properties": [
```

```
                    {
                      "id": "custom.axisPlacement",
                      "value": "right"
                    },
                    {
                      "id": "custom.drawStyle",
                      "value": "bars"
                    },
                    {
                      "id": "custom.fillOpacity",
                      "value": 30
                    },
                    {
                      "id": "min",
                      "value": 0
                    },
                    {
                      "id": "max",
                      "value": 1.2
                    },
                    {
                      "id": "custom.axisLabel",
                      "value": "Fault Active (1 = Yes)"
                    }
                  ]
                }
              ]
            },
            "pluginVersion": "12.4.0",
            "targets": [
              {
                "refId": "A",
                "format": "time_series",
                "rawQuery": true,
                "editorMode": "code",
                "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.bacnet_device_id::text IN ($
{device:sqlstring})\n  AND p.external_id IN (${point:sqlstring})
\nGROUP BY 1, 3\nORDER BY 1, 3;"
              },
              {
                "refId": "B",
                "format": "time_series",
                "rawQuery": true,
                "editorMode": "code",
                "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), fr.ts) AS \"time\",\n
max(fr.flag_value) AS \"value\",\n  fd.name AS \"metric\"\nFROM
```

```
fault_results fr\nJOIN fault_definitions fd ON fd.fault_id =
fr.fault_id\nWHERE $__timeFilter(fr.ts)\n  AND (fr.site_id IN ($
{site:sqlstring}) OR fr.site_id IN (SELECT name FROM sites WHERE
id::text IN (${site:sqlstring})))\nGROUP BY 1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "multi",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

42.3 Host stats (stack-host-stats)

The **stack-host-stats** container scrapes host memory, load, swap, disk usage, and Docker container stats into `host_metrics`, `disk_metrics`, and `container_metrics`. Use the same TimescaleDB datasource (`openfdd_timescale`).

42.3.1 Dashboard variable (Host)

- **Name:** `hostname`
- **Type:** Query
- **Datasource:** openfdd_timescale
- **Query:**

```sql
SELECT DISTINCT hostname AS __text, hostname AS __value
FROM host_metrics
WHERE $__timeFilter(ts)
ORDER BY 1;
```

42.3.2 Host — Memory (used / available / total)

```
        {
          "id": null,
```

```
          "type": "timeseries",
          "title": "Host — Memory (used / available / total)",
          "gridPos": { "x": 0, "y": 0, "h": 10, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "drawStyle": "line",
                "lineInterpolation": "linear",
                "lineWidth": 2,
                "fillOpacity": 12,
                "axisPlacement": "left",
                "scaleDistribution": { "type": "linear" }
              },
              "unit": "bytes",
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              {
                "matcher": { "id": "byName", "options":
"mem_used_bytes" },
                "properties": [{ "id": "displayName", "value": "Used" }]
              },
              {
                "matcher": { "id": "byName", "options":
"mem_available_bytes" },
                "properties": [{ "id": "displayName", "value":
"Available" }]
              },
              {
                "matcher": { "id": "byName", "options":
"mem_total_bytes" },
                "properties": [{ "id": "displayName", "value": "Total" }]
              }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", mem_used_bytes AS
\"mem_used_bytes\", mem_available_bytes AS
\"mem_available_bytes\", mem_total_bytes AS \"mem_total_bytes\"
FROM host_metrics WHERE $__timeFilter(ts) AND hostname IN ($
{hostname:sqlstring}) ORDER BY ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
```

```
"bottom" } }
        }
```

42.3.3 Host — Load (1 / 5 / 15 min)

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Load (1 / 5 / 15 min)",
          "gridPos": { "x": 0, "y": 10, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 8, "axisPlacement": "left" },
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options": "load_1" },
"properties": [{ "id": "displayName", "value": "Load 1m" }] },
              { "matcher": { "id": "byName", "options": "load_5" },
"properties": [{ "id": "displayName", "value": "Load 5m" }] },
              { "matcher": { "id": "byName", "options": "load_15" },
"properties": [{ "id": "displayName", "value": "Load 15m" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", load_1 AS \"load_1\",
load_5 AS \"load_5\", load_15 AS \"load_15\" FROM host_metrics
WHERE $__timeFilter(ts) AND hostname IN (${hostname:sqlstring})
ORDER BY ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

42.3.4 Host — Swap (used / total)

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Swap (used / total)",
```

```
          "gridPos": { "x": 0, "y": 18, "h": 6, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 10, "axisPlacement": "left" },
              "unit": "bytes",
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options":
"swap_used_bytes" }, "properties": [{ "id": "displayName",
"value": "Swap used" }] },
              { "matcher": { "id": "byName", "options":
"swap_total_bytes" }, "properties": [{ "id": "displayName",
"value": "Swap total" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", swap_used_bytes AS
\"swap_used_bytes\", swap_total_bytes AS
\"swap_total_bytes\" FROM host_metrics WHERE $__timeFilter(ts) AND
hostname IN (${hostname:sqlstring}) ORDER BY ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

42.3.5 Host — Disk space (free / used / total)

Disk metrics come from `disk_metrics` (mount path from env `OFDD_DISK_MOUNT_PATHS`, default `/`). The table is created by the same migration as `host_metrics` (`stack/sql/006_host_metrics.sql`); re-run migrations if you added host-stats after an older deploy.

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Disk space (free / used / total)",
          "gridPos": { "x": 0, "y": 24, "h": 10, "w": 24 },
          "fieldConfig": {
            "defaults": {
```

```
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 12, "axisPlacement": "left" },
              "unit": "bytes",
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options": "free_bytes" },
"properties": [{ "id": "displayName", "value": "Free" }] },
              { "matcher": { "id": "byName", "options": "used_bytes" },
"properties": [{ "id": "displayName", "value": "Used" }] },
              { "matcher": { "id": "byName", "options": "total_bytes" },
"properties": [{ "id": "displayName", "value": "Total" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", free_bytes AS
\"free_bytes\", used_bytes AS \"used_bytes\", total_bytes AS
\"total_bytes\" FROM disk_metrics WHERE $__timeFilter(ts) AND
hostname IN (${hostname:sqlstring}) AND mount_path = '/' ORDER BY
ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

42.3.6 Host — Disk available (gauge, latest)

```
        {
          "id": null,
          "type": "gauge",
          "title": "Host — Disk space available (latest)",
          "gridPos": { "x": 0, "y": 34, "h": 6, "w": 8 },
          "fieldConfig": {
            "defaults": {
              "unit": "bytes",
              "min": 0,
              "color": { "mode": "thresholds" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"red", "value": null}, {"color": "yellow", "value": 5368709120},
{"color": "green", "value": 10737418240}] }
            }
          },
```

```
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", free_bytes AS \"value\"
FROM disk_metrics WHERE hostname IN (${hostname:sqlstring}) AND
mount_path = '/' ORDER BY ts DESC LIMIT 1;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "showThresholdLabels": false }
        }
```

42.3.7 Containers — CPU %

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Containers — CPU %",
          "gridPos": { "x": 0, "y": 40, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 1,
"fillOpacity": 5, "axisPlacement": "left" },
              "unit": "percent",
              "min": 0,
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            }
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", cpu_pct AS \"value\",
container_name AS \"metric\" FROM container_metrics WHERE
$__timeFilter(ts) ORDER BY ts, container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

42.3.8 Containers — Memory %

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Containers — Memory %",
          "gridPos": { "x": 0, "y": 48, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 1,
"fillOpacity": 5, "axisPlacement": "left" },
              "unit": "percent",
              "min": 0,
              "max": 100,
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            }
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", COALESCE(mem_pct, 0) AS
\"value\", container_name AS \"metric\" FROM container_metrics
WHERE $__timeFilter(ts) ORDER BY ts, container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

42.3.9 Containers — Table (latest)

```
        {
          "id": null,
          "type": "table",
          "title": "Containers — Latest stats",
          "gridPos": { "x": 0, "y": 56, "h": 12, "w": 24 },
          "fieldConfig": {
            "defaults": { "custom": { "align": "auto", "displayMode":
"auto" }, "mappings": [], "thresholds": { "mode": "absolute",
"steps": [{"color": "green", "value": null}] } },
            "overrides": [
              { "matcher": { "id": "byName", "options": "cpu_pct" },
"properties": [{ "id": "unit", "value": "percent" }] },
              { "matcher": { "id": "byName", "options": "mem_pct" },
"properties": [{ "id": "unit", "value": "percent" }] },
```

```
              { "matcher": { "id": "byName", "options":
"mem_usage_bytes" }, "properties": [{ "id": "unit", "value":
"bytes" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "table",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT container_name, ts, cpu_pct, mem_pct,
mem_usage_bytes, pids FROM (SELECT *, row_number() OVER (PARTITION
BY container_name ORDER BY ts DESC) AS rn FROM container_metrics
WHERE $__timeFilter(ts)) sub WHERE rn = 1 ORDER BY
container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "showHeader": true }
        }
```

42.4 Weather

42.4.1 Temp RH Dewpoint

```
        {
          "id": 21,
          "type": "timeseries",
          "title": "Weather — Temp / RH / Dewpoint",
          "gridPos": {
            "x": 0,
            "y": 0,
            "h": 10,
            "w": 24
          },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "drawStyle": "line",
                "lineInterpolation": "linear",
                "barAlignment": 0,
                "barWidthFactor": 0.6,
                "lineWidth": 2,
                "fillOpacity": 12,
                "gradientMode": "none",
                "spanNulls": false,
                "insertNulls": false,
                "showPoints": "never",
```

```
            "showValues": false,
            "pointSize": 4,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "°F",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": false,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "color": "green",
                "value": null
              },
              {
                "color": "red",
                "value": 80
              }
            ]
          }
        },
        "overrides": [
          {
            "matcher": {
              "id": "byName",
              "options": "temp_f"
            },
            "properties": [
              {
                "id": "unit",
                "value": "fahrenheit"
              },
```

```
            {
              "id": "custom.axisPlacement",
              "value": "left"
            },
            {
              "id": "custom.axisLabel",
              "value": "°F"
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "dewpoint_f"
          },
          "properties": [
            {
              "id": "unit",
              "value": "fahrenheit"
            },
            {
              "id": "custom.axisPlacement",
              "value": "left"
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "rh_pct"
          },
          "properties": [
            {
              "id": "unit",
              "value": "percent"
            },
            {
              "id": "custom.axisPlacement",
              "value": "right"
            },
            {
              "id": "custom.axisLabel",
              "value": "RH (%)"
            },
            {
              "id": "min",
              "value": 0
            },
            {
              "id": "max",
              "value": 100
            }
```

```
                ]
              }
            ]
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "refId": "A",
              "editorMode": "code",
              "rawQuery": true,
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('temp_f','rh_pct','dewpoint_f')\nGROUP BY 1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```


42.4.2 Wind Speed Gusts Direction


```
        {
          "id": 22,
          "type": "timeseries",
          "title": "Weather — Wind (Speed / Gust / Direction)",
          "gridPos": {
            "x": 0,
            "y": 10,
            "h": 9,
            "w": 24
          },
```

```
"fieldConfig": {
  "defaults": {
    "custom": {
      "drawStyle": "line",
      "lineInterpolation": "linear",
      "barAlignment": 0,
      "barWidthFactor": 0.6,
      "lineWidth": 2,
      "fillOpacity": 10,
      "gradientMode": "none",
      "spanNulls": false,
      "insertNulls": false,
      "showPoints": "never",
      "showValues": false,
      "pointSize": 4,
      "stacking": {
        "mode": "none",
        "group": "A"
      },
      "axisPlacement": "left",
      "axisLabel": "mph",
      "axisColorMode": "text",
      "axisBorderShow": false,
      "scaleDistribution": {
        "type": "linear"
      },
      "axisCenteredZero": false,
      "hideFrom": {
        "tooltip": false,
        "viz": false,
        "legend": false
      },
      "thresholdsStyle": {
        "mode": "off"
      }
    },
    "color": {
      "mode": "palette-classic"
    },
    "mappings": [],
    "thresholds": {
      "mode": "absolute",
      "steps": [
        {
          "color": "green",
          "value": null
        },
        {
          "color": "red",
          "value": 80
        }
      ]
```

```
      }
    },
    "overrides": [
      {
        "matcher": {
          "id": "byName",
          "options": "wind_mph"
        },
        "properties": [
          {
            "id": "unit",
            "value": "mph"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "custom.axisLabel",
            "value": "mph"
          },
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "gust_mph"
        },
        "properties": [
          {
            "id": "unit",
            "value": "mph"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "wind_dir_deg"
        },
```

```
                "properties": [
                  {
                    "id": "unit",
                    "value": "degrees"
                  },
                  {
                    "id": "custom.axisPlacement",
                    "value": "right"
                  },
                  {
                    "id": "custom.axisLabel",
                    "value": "°"
                  },
                  {
                    "id": "min",
                    "value": 0
                  },
                  {
                    "id": "max",
                    "value": 360
                  }
                ]
              }
            ]
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "refId": "A",
              "editorMode": "code",
              "rawQuery": true,
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('wind_mph','gust_mph','wind_dir_deg')\nGROUP BY 1,
3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
```

```
        "showLegend": true,
        "displayMode": "list",
        "placement": "bottom",
        "calcs": []
      }
    }
  }
```

42.4.3 Wind Speed Gust Direction

```
  {
    "id": 22,
    "type": "timeseries",
    "title": "Weather — Wind (Speed / Gust / Direction)",
    "gridPos": {
      "x": 0,
      "y": 10,
      "h": 9,
      "w": 24
    },
    "fieldConfig": {
      "defaults": {
        "custom": {
          "drawStyle": "line",
          "lineInterpolation": "linear",
          "barAlignment": 0,
          "barWidthFactor": 0.6,
          "lineWidth": 2,
          "fillOpacity": 10,
          "gradientMode": "none",
          "spanNulls": false,
          "insertNulls": false,
          "showPoints": "never",
          "showValues": false,
          "pointSize": 4,
          "stacking": {
            "mode": "none",
            "group": "A"
          },
          "axisPlacement": "left",
          "axisLabel": "mph",
          "axisColorMode": "text",
          "axisBorderShow": false,
          "scaleDistribution": {
            "type": "linear"
          },
          "axisCenteredZero": false,
          "hideFrom": {
            "tooltip": false,
            "viz": false,
            "legend": false
```

```
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "color": "green",
                "value": null
              },
              {
                "color": "red",
                "value": 80
              }
            ]
          }
        },
        "overrides": [
          {
            "matcher": {
              "id": "byName",
              "options": "wind_mph"
            },
            "properties": [
              {
                "id": "unit",
                "value": "mph"
              },
              {
                "id": "custom.axisPlacement",
                "value": "left"
              },
              {
                "id": "custom.axisLabel",
                "value": "mph"
              },
              {
                "id": "min",
                "value": 0
              }
            ]
          },
          {
            "matcher": {
              "id": "byName",
              "options": "gust_mph"
```

```
        },
        "properties": [
          {
            "id": "unit",
            "value": "mph"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "wind_dir_deg"
        },
        "properties": [
          {
            "id": "unit",
            "value": "degrees"
          },
          {
            "id": "custom.axisPlacement",
            "value": "right"
          },
          {
            "id": "custom.axisLabel",
            "value": "°"
          },
          {
            "id": "min",
            "value": 0
          },
          {
            "id": "max",
            "value": 360
          }
        ]
      }
    ]
  },
  "pluginVersion": "12.4.0",
  "targets": [
    {
      "refId": "A",
      "editorMode": "code",
      "rawQuery": true,
```

```
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('wind_mph','gust_mph','wind_dir_deg')\nGROUP BY 1,
3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

42.4.4 Solar Radiation

```
        {
          "id": 23,
          "type": "timeseries",
          "title": "Weather — Solar / Radiation (W/m²)",
          "gridPos": {
            "x": 0,
            "y": 19,
            "h": 9,
            "w": 24
          },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "drawStyle": "line",
                "lineInterpolation": "linear",
                "barAlignment": 0,
                "barWidthFactor": 0.6,
                "lineWidth": 2,
                "fillOpacity": 10,
                "gradientMode": "none",
```

```
            "spanNulls": false,
            "insertNulls": false,
            "showPoints": "never",
            "showValues": false,
            "pointSize": 4,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "W/m²",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": true,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "color": "green",
                "value": null
              },
              {
                "color": "red",
                "value": 80
              }
            ]
          },
          "unit": "wattperm2",
          "min": 0
        },
        "overrides": [
          {
            "matcher": {
              "id": "byName",
              "options": "shortwave_wm2"
            },
```

```
          "properties": [
            {
              "id": "min",
              "value": 0
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "direct_wm2"
          },
          "properties": [
            {
              "id": "min",
              "value": 0
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "diffuse_wm2"
          },
          "properties": [
            {
              "id": "min",
              "value": 0
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "gti_wm2"
          },
          "properties": [
            {
              "id": "min",
              "value": 0
            }
          ]
        }
      ]
    },
    "pluginVersion": "12.4.0",
    "targets": [
      {
        "refId": "A",
        "editorMode": "code",
        "rawQuery": true,
        "format": "time_series",
```

```
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('shortwave_wm2','direct_wm2','diffuse_wm2','gti_wm2')\nGROUP BY
1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

42.4.5 Cloud Cover

```
        {
          "id": 24,
          "type": "timeseries",
          "title": "Weather — Cloud Cover (%)",
          "gridPos": {
            "x": 0,
            "y": 28,
            "h": 7,
            "w": 24
          },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "drawStyle": "line",
                "lineInterpolation": "linear",
                "barAlignment": 0,
                "barWidthFactor": 0.6,
                "lineWidth": 2,
                "fillOpacity": 35,
                "gradientMode": "opacity",
```

```
          "spanNulls": false,
          "insertNulls": false,
          "showPoints": "never",
          "showValues": false,
          "pointSize": 4,
          "stacking": {
            "mode": "none",
            "group": "A"
          },
          "axisPlacement": "left",
          "axisLabel": "Cloud (%)",
          "axisColorMode": "text",
          "axisBorderShow": false,
          "scaleDistribution": {
            "type": "linear"
          },
          "axisCenteredZero": false,
          "hideFrom": {
            "tooltip": false,
            "viz": false,
            "legend": false
          },
          "thresholdsStyle": {
            "mode": "off"
          }
        },
        "color": {
          "mode": "palette-classic"
        },
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [
            {
              "color": "green",
              "value": null
            },
            {
              "color": "red",
              "value": 80
            }
          ]
        },
        "unit": "percent",
        "min": 0,
        "max": 100
      },
      "overrides": []
    },
    "pluginVersion": "12.4.0",
    "targets": [
      {
```

```
              "refId": "A",
              "editorMode": "code",
              "rawQuery": true,
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN ('cloud_pct')\nGROUP
BY 1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

42.5 Faults

42.5.1 Fault Definitions

```
        {
          "id": 1,
          "type": "table",
          "title": "Fault Definitions",
          "gridPos": {
            "x": 0,
            "y": 0,
            "h": 10,
            "w": 14
          },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "align": "auto",
```

```
                "footer": {
                  "reducers": []
                },
                "cellOptions": {
                  "type": "auto"
                },
                "inspect": false,
                "hideFrom": {
                  "viz": false
                }
              },
              "mappings": [],
              "thresholds": {
                "mode": "absolute",
                "steps": [
                  {
                    "color": "green",
                    "value": null
                  },
                  {
                    "color": "red",
                    "value": 80
                  }
                ]
              },
              "color": {
                "mode": "thresholds"
              }
            },
            "overrides": []
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "dataset": "openfdd",
              "datasource": {
                "type": "grafana-postgresql-datasource",
                "uid": "openfdd_timescale"
              },
              "editorMode": "code",
              "format": "table",
              "rawQuery": true,
              "rawSql": "SELECT \r\n  fault_id, \r\n  name, \r\n
category, \r\n  severity,
\r\n  -- Shows what Brick equipment this rule binds to (e.g., AHU,
VAV)\r\n  array_to_string(equipment_types, ', ') AS
target_equipment,
\r\n  -- Shows the raw JSON of the specific Brick points it needs
(e.g., sat, mat)\r\n  inputs AS required_points,\r\n
updated_at\r\nFROM fault_definitions\r\nWHERE
$__timeFilter(updated_at)\r\nORDER BY updated_at DESC;",
              "refId": "A",
```

```
        "sql": {
          "columns": [
            {
              "parameters": [],
              "type": "function"
            }
          ],
          "groupBy": [
            {
              "property": {
                "type": "string"
              },
              "type": "groupBy"
            }
          ],
          "limit": 50
        }
      }
    ],
    "datasource": {
      "type": "grafana-postgresql-datasource",
      "uid": "openfdd_timescale"
    },
    "options": {
      "showHeader": true,
      "cellHeight": "sm"
    }
  }
```

42.5.2 Fault Definition Count

```
  {
    "id": 2,
    "type": "stat",
    "title": "Fault Definition Count",
    "gridPos": {
      "x": 0,
      "y": 15,
      "h": 15,
      "w": 24
    },
    "fieldConfig": {
      "defaults": {
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [
            {
              "color": "green",
              "value": null
```

```
                    },
                    {
                      "color": "red",
                      "value": 80
                    }
                  ]
                },
                "color": {
                  "mode": "thresholds"
                }
              },
              "overrides": []
            },
            "pluginVersion": "12.4.0",
            "targets": [
              {
                "dataset": "openfdd",
                "datasource": {
                  "type": "grafana-postgresql-datasource",
                  "uid": "openfdd_timescale"
                },
                "editorMode": "code",
                "format": "table",
                "rawQuery": true,
                "rawSql": "SELECT count(*)::int AS \"Fault
definitions\"\r\nFROM fault_definitions;",
                "refId": "A",
                "sql": {
                  "columns": [
                    {
                      "parameters": [],
                      "type": "function"
                    }
                  ],
                  "groupBy": [
                    {
                      "property": {
                        "type": "string"
                      },
                      "type": "groupBy"
                    }
                  ],
                  "limit": 50
                }
              }
            ],
            "datasource": {
              "type": "grafana-postgresql-datasource",
              "uid": "openfdd_timescale"
            },
            "options": {
              "reduceOptions": {
```

```
          "values": false,
          "calcs": [
            "lastNotNull"
          ],
          "fields": ""
        },
        "orientation": "auto",
        "textMode": "auto",
        "wideLayout": true,
        "colorMode": "value",
        "graphMode": "area",
        "justifyMode": "auto",
        "showPercentChange": false,
        "percentChangeColorMode": "standard"
      }
    }
```

43 Grafana dashboards

44 Grafana dashboards (optional)

Grafana is optional. Install with `./scripts/bootstrap.sh --with-grafana`. The React frontend provides equivalent timeseries, fault, and system-resource views. Use Grafana when you want custom SQL dashboards or existing Grafana workflows.

No dashboards are provisioned by default. Only the **datasource** (TimescaleDB, uid: `openfdd_timescale`) is prebuilt.

To build your own dashboards using SQL and the Open-FDD database, see the **Grafana SQL cookbook**.

You can also place dashboard JSON files in `stack/grafana/dashboards/` (or the directory mounted at `/var/lib/grafana/dashboards` in the Grafana container) and restart Grafana to load them.

44.1 React frontend parity

The React UI provides equivalent time-series and stat views for FDD workflows:

- **Trending** — Multi-point time-series (site/equipment/point picker, date range, dual Y-axis). Uses the same `timeseries_readings` data as the Grafana BACnet timeseries dashboard.
- **Faults** — Active fault table (with **Device** and **Sensor/point** columns so you can see which bad sensor was found on which device), fault definitions, fault count (period) stat, and fault flags over time chart (from `fault_results` via `GET /analytics/fault-timeseries` and `GET /analytics/fault-summary`).
- **Plots (trend + fault overlay)** — Same idea as the Grafana "BACnet Plus Fault Data" panel: one chart with point time-series (left axis) and fault data overlaid. In the React UI, select a site, add points and **Add faults**; fault activity is drawn as **vertical colored bands** on the same plot (data from `GET /analytics/fault-timeseries`, i.e. `fault_results`). So fault data does show on the plot in parity with the Grafana version.
- **Overview** — Site cards with fault counts, equipment/points stats, and FDD status.
- **System resources** — A dedicated sidebar page that mirrors the Grafana "System Resources (Host & Containers)" dashboard: host memory used/available, load average (1m/5m/15m), swap used (stat cards with threshold colors), host memory and load time-series charts, container memory (MB) and CPU % charts, **Containers (latest)** table (CPU %, Mem MB, Mem %, PIDs), and **disk usage** per mount (used/free/total GB, % used, progress bar). Data comes from `host_metrics`, `container_metrics`, and `disk_metrics` via `GET /analytics/system/*`. Requires the **host-stats** service to be running so those tables are populated.

If you prefer building custom SQL dashboards in Grafana, use the cookbook and the same tables (`host_metrics`, `container_metrics`, `disk_metrics`, `timeseries_readings`, `fault_results`, etc.).

45 Configuration

46 Configuration

46.1 Platform config (RDF + CRUD)

Platform config lives in the **same RDF graph** as Brick and BACnet (`config/data_model.ttl`). No YAML file. **Where:** Bootstrap seeds via PUT /config; API GET/PUT /config and POST /data-model/sparql use the graph.

- **Bootstrap:** `./scripts/bootstrap.sh` seeds config via PUT /config (defaults or `OFDD_*` from `stack/.env`).
- **API:** GET /config, PUT /config; query via POST /data-model/sparql.
- **Env at seed:** Set `OFDD_*` in `stack/.env` before first bootstrap to customize the seed. **Docker:** Set env in `stack/docker-compose.yml` or `stack/.env`; bootstrap uses these when calling PUT /config.

Rename / multiple configs: Use different env files (e.g. `platform-prod.env`, `platform-building-a.env`) and start with `docker compose --env-file platform-building-a.env up -d`, or set `OFDD_*` in that file. No built-in "config name" selector; use env files or env vars per deployment.

46.2 Platform keys (RDF / GET-PUT /config)

Example keys (GET/PUT /config or OFDD_* at bootstrap seed):

Key	Default	Description
`rule_interval_hours`	3	FDD loop run interval (when no trigger file: run every N hours)
`lookback_days`	3	Days of data loaded per run (each run pulls last N days into the rule engine)
`rules_dir`	analyst/rules	**Single directory for your rules** (hot reload)

Key	Default	Description
`brick_ttl_dir`	—	Optional. Directory containing Brick model TTL (e.g. `config/`); platform uses first `.ttl` or brick_ttl path for FDD column mapping. Optional if using points `brick_type`/fdd_input. See Data modeling.
`bacnet_enabled`	true	Enable BACnet scraper
`graph_sync_interval_min`	5	Minutes between serializing the in-memory graph to `data_model.ttl` (API background thread). Env: `OFDD_GRAPH_SYNC_INTERVAL_MIN`.
`bacnet_scrape_interval_min`	5	Poll interval (minutes)
`bacnet_site_id`	default	Site to tag when scraping (use on **remote gateways** so data is attributed to the right building on the central DB)
`bacnet_gateways`	—	Optional. **Central aggregator:** JSON array of `{url, site_id}`; scraper polls each remote diy-bacnet-server in turn. Env: `OFDD_BACNET_GATEWAYS`.
`open_meteo_enabled`	true	Enable weather; when true, **FDD loop runs a weather fetch at the start of each run** (same cadence as rules, every `rule_interval_hours`).
`open_meteo_interval_hours`	24	**Standalone weather-scraper only.** Poll interval (hours) when using the optional weather-scraper container. Ignored when weather is run from the FDD loop.
`open_meteo_latitude`	41.88	Site latitude
`open_meteo_longitude`	-87.63	Site longitude
`open_meteo_timezone`	America/Chicago	Timezone
`open_meteo_days_back`	3	**Standalone weather-scraper only.** Days of archive to fetch

Key	Default	Description
		per run. When weather runs with FDD, a 1-day lookback is used (no separate config).
`open_meteo_site_id`	default	Site ID for weather points

Weather fetch: Weather is normally fetched **with each FDD run** (same interval as fault rules, e.g. every 3 h), using a **1-day lookback** so the API is not over-used. A standalone weather scraper is only for setups that do not run the FDD loop; do not run both to avoid redundant fetches. The standalone scraper reads config from **GET /config** (like the BACnet scraper) when available.

Where to place rules: Put your project rules in **`analyst/rules/`** (one place). See Fault rules overview and the Expression Rule Cookbook.

Hot reload (AFDD tuning): The project supports **hot reloading of YAML rule files** so the AFDD maintainer can make tweaks and adjustments for fault tuning without restarting the platform. The FDD loop loads rules from `rules_dir` on every run; edit YAML → trigger a run (or wait for the schedule) → results reflect the new params. The `rules_dir` path itself is **RDF-driven**: it comes from platform config (GET `/config` or the same graph that backs PUT `/config`). Unit tests that cover this behaviour: `open_fdd/tests/platform/test_rules_loader.py` (hot-reload hash and reload-on-change) and `open_fdd/tests/platform_api/test_rules.py` (list/read rule files from configured `rules_dir`). A future CRUD-driven rule editor in the frontend (with the same hot-reload semantics) would be a possible evolution; today, YAML in `analyst/rules` is the supported path.

Rolling window (per rule): Set `params.rolling_window` in each rule YAML; see Fault rules for HVAC.

46.3 Services that read config from the API (BACnet scraper)

Platform config (e.g. **Scrape interval (min)**) is stored in the data model and served by **GET /config**. Some services need to call the API to get that config so changes in the Config UI take effect.

BACnet scraper and **weather scraper (standalone):** Both can read config from **GET /config** (with a short cache) so the Config UI and data model control their behaviour. The BACnet scraper uses `bacnet_scrape_interval_min`; the standalone weather scraper uses `open_meteo_enabled`, `open_meteo_interval_hours`, `open_meteo_days_back`, and the geo/site fields. If GET /config fails (e.g. no `OFDD_API_KEY`), they fall back to env vars.

BACnet scraper: The scraper runs on a fixed interval (e.g. every 1, 5, or 10 minutes). It can get that interval in two ways:

1. **From the API (dynamic)** — The scraper calls **GET /config** (with a short cache). It then uses `bacnet_scrape_interval_min` from the response, so whatever you set in the Config UI is what the scraper uses. **This only works when the scraper can authenticate:** the API requires Bearer auth when `OFDD_API_KEY` is set, so the scraper must have **`OFDD_API_KEY`** in its environment (same value as in `stack/.env`). The stack's `docker-compose.yml` passes `OFDD_API_KEY: ${OFDD_API_KEY:-}` to the bacnet-scraper service for this reason.
2. **Fallback (env)** — If GET /config fails (e.g. 401 because no API key, or API unreachable), the scraper falls back to **`OFDD_BACNET_SCRAPE_INTERVAL_MIN`** from its environment (e.g. 5 in compose). **So if you set an interval in the Config UI but the scraper does not have `OFDD_API_KEY`, it will ignore the UI and keep using the env default.** Rebuild/restart the bacnet-scraper after ensuring `OFDD_API_KEY` is in `stack/.env` and in the scraper's env: `./scripts/bootstrap.sh --build bacnet-scraper`.

Summary: For the Config UI (and data model) to control the BACnet scrape interval, the API must have `OFDD_API_KEY` set and the bacnet-scraper container must receive the same key so it can call GET /config successfully.

46.4 BACnet: single gateway, remote gateways, central aggregator

- **Single building (or one remote gateway):** Set `OFDD_BACNET_SERVER_URL` and optionally `OFDD_BACNET_SITE_ID` (e.g. `building-a`). The scraper tags all readings with that site. On a **remote** gateway (diy-bacnet-server + scraper on another subnet), point `OFDD_DB_DSN` at the central Open-FDD DB and set `OFDD_BACNET_SITE_ID` to the site name/UUID used on the central API so data is attributed to that building.

- **Central aggregator (multiple remote gateways):** On the central host, do **not** run local bacnet-server/bacnet-scraper; run only DB, API, Grafana, FDD loop, and (optional) weather. Set `OFDD_BACNET_GATEWAYS` to a JSON array, e.g. `[{"url":"http://10.1.1.1:8080","site_id":"building-a"},{"url":"http://10.1.2.1:8080","site_id":"building-b"}]`, and run one scraper container (or cron) that polls each URL and writes to the central DB with the given `site_id`. Alternatively, deploy one scraper per building elsewhere, each with `OFDD_DB_DSN=central` and `OFDD_BACNET_SITE_ID=<that building>`.

Open-Meteo weather points (stored in `timeseries_readings`, `external_id`):

Point	Description
`temp_f`	Temperature (°F)
`rh_pct`	Relative humidity (%)
`dewpoint_f`	Dew point (°F)
`wind_mph`	Wind speed (mph)
`gust_mph`	Wind gusts (mph)
`wind_dir_deg`	Wind direction (degrees)
`shortwave_wm2`	Shortwave radiation (W/m²)
`direct_wm2`	Direct radiation (W/m²)
`diffuse_wm2`	Diffuse radiation (W/m²)
`gti_wm2`	Global tilted irradiance (W/m²)
`cloud_pct`	Cloud cover (%)

In the **data model** and **Points** UI these appear under equipment **Open-Meteo** (type **Weather_Service**); the RDF graph marks that equipment with `ofdd:dataSource "open_meteo"`.

46.5 Edge / resource limits

For edge deployments with limited disk, set these at bootstrap (or in `stack/.env`). See Getting Started — Bootstrap options.

Setting	Default	Bootstrap arg	Env (stack/.env)
Data retention	365 days	`--retention-days N`	`OFDD_RETENTION_DAYS`

Setting	Default	Bootstrap arg	Env (stack/.env)
Docker log size	100m per file	`--log-max-size SIZE`	`OFDD_LOG_MAX_SIZE`
Docker log files	3	`--log-max-files N`	`OFDD_LOG_MAX_FILES`

Data retention: TimescaleDB drops chunks older than the configured interval from `timeseries_readings`, `fault_results`, `host_metrics`, `container_metrics`. Set at first run, e.g. `./scripts/bootstrap.sh --retention-days 180`, or set `OFDD_RETENTION_DAYS` in `stack/.env` before running bootstrap.

Log rotation: Containers use `json-file` driver; size and file count come from the table above. Restart the stack after changing so new values apply.

Accessing logs: `docker logs <container> --tail 50` (e.g. `openfdd_weather_scraper`, `openfdd_bacnet_scraper`).

46.6 Environment

`OFDD_`-prefixed vars override YAML:

Variable	Description
`OFDD_DB_HOST`	TimescaleDB host
`OFDD_DB_PORT`	TimescaleDB port
`OFDD_DB_NAME`	Database name
`OFDD_DB_USER`	Database user
`OFDD_DB_PASSWORD`	Database password
`OFDD_BACNET_SERVER_URL`	diy-bacnet-server base URL (e.g. http://localhost:8080)
`OFDD_BACNET_SITE_ID`	Site to tag when scraping (default: default; use on remote gateways)
`OFDD_BACNET_GATEWAYS`	JSON array of {url, site_id} for central aggregator
`OFDD_RULES_DIR`	Rules directory (default: analyst/rules)

47 Contributing

48 Contributing to Open-FDD

First off, thanks for taking the time to contribute!

Open-FDD is in **Alpha**. The most valuable contributions right now are **bug reports** and **FDD rules**—especially from mechanical engineers and building professionals who can add and refine fault-detection rules using the Expression Rule Cookbook. All types of contributions are encouraged.

Phase focus: Alpha emphasizes platform stability, driver implementation beyond BACnet, and API changes for specific integration needs. Beta (planned) will focus more on Brick data modeling (e.g. ASHRAE 223P), mechanical engineering and consulting input into the expression rule cookbook, and better default Grafana dashboards for HVAC analytics. See the Table of Contents for different ways to help and how this project handles them. Please read the relevant section before contributing; it helps maintainers and keeps things smooth for everyone.

> If you like the project but don't have time to contribute, that's fine. Other ways to support it: - Star the repository - Share it with colleagues or at meetups - Refer to Open-FDD in your project's readme or documentation

---

48.1 Table of Contents

- Code of Conduct
- I Have a Question
- I Want To Contribute
- Reporting Bugs
- Suggesting Enhancements
- Contributing Rules (Expression Rule Cookbook)
- Your First Code Contribution
- Improving the Documentation
- Styleguides
- Git workflow (branch → PR → sync)
- Commit Messages

- Join the Project Team

48.2 Code of Conduct

This project expects everyone to be respectful and constructive. By participating, you agree to uphold a positive environment. Please report unacceptable behavior by opening an issue or contacting the repository maintainers.

48.3 I Have a Question

Before asking, please check the documentation and search existing issues to see if your question is already answered.

If you still need help:

- Open an issue.
- Provide as much context as you can (what you're trying to do, what you ran, what happened).
- Include relevant versions: Python, Docker, OS (e.g. Ubuntu, Linux Mint), and Open-FDD commit or release if known.

We'll respond as soon as we can.

48.4 I Want To Contribute

48.4.1 Legal notice

When contributing, you agree that you have authored 100% of the content, have the necessary rights to it, and that your contribution may be provided under the project license (MIT).

48.5 Reporting Bugs

Bug testing is a top priority in Alpha. Good bug reports help us fix issues quickly.

48.5.1 Before submitting a bug report

- Use the latest version (main branch or latest release).
- Confirm the bug is in Open-FDD and not in your environment (e.g. wrong Python version, missing config). Check the documentation and I Have a Question first.
- Search issues to see if the bug is already reported.
- Collect:
  - **Stack trace** (Traceback) if applicable
  - **OS and platform** (e.g. Linux, macOS, Windows; x86, ARM)
  - **Versions**: Python, Docker, Docker Compose, and how you ran Open-FDD (pip, Docker Compose, etc.)
  - **Steps to reproduce** and, if possible, sample input/config
  - Whether you can reproduce it every time and on an older version

48.5.2 How to submit a good bug report

- **Security issues:** Do **not** report security vulnerabilities in public issues. Email the repository owner or open a private security advisory on GitHub.
- Open a new issue. Don't assume it's a bug yet—avoid using the word "bug" in the title until it's confirmed.
- Describe **expected behavior** vs **actual behavior**.
- Provide **reproduction steps** so someone else can recreate the issue. Isolate the problem when possible (e.g. minimal rule YAML, minimal config).
- Paste the information you collected above.

After you file:

- Maintainers will label the issue. If we can't reproduce it, we may ask for more steps and mark it `needs-repro`. Reproducible issues may be marked `needs-fix` and opened for implementation.

48.6 Suggesting Enhancements

Enhancements are tracked as GitHub issues.

48.6.1 Before submitting

- Use the latest version and read the documentation to see if the behavior already exists or can be configured.
- Search issues to see if the enhancement was already suggested; if so, add to that discussion.
- Consider whether the idea fits Open-FDD's scope (edge AFDD, rules, API, BACnet, Grafana). Make a clear case for why it would help most users.

48.6.2 How to submit a good enhancement suggestion

- Use a **clear, descriptive title**.
- Describe the **current behavior** and what you **want instead**, and why.
- Provide **step-by-step detail** where useful; screenshots or example YAML/config can help.
- Explain why this would be useful to the community.

48.7 Contributing Rules (Expression Rule Cookbook)

We especially welcome contributions from mechanical engineers and building professionals who can add or improve FDD rules.

- **Where rules live:** Fault rules overview — put project rules in `analyst/rules/` (YAML). The FDD loop reloads them every run; no restart needed.
- **How to write rules:** Expression Rule Cookbook — expression-type rules use YAML with BRICK-style inputs, params, and pandas/NumPy expressions. The cookbook includes AHU-style rules (e.g. GL36-inspired) and patterns you can adapt.
- **What to contribute:**
  - **New rule YAMLs** for common faults (AHU, VAV, plant, sensors) that others can reuse.
  - **Improvements to existing cookbook rules** (thresholds, logic, descriptions).
  - **New cookbook sections or examples** (e.g. for a specific equipment type or standard).

To contribute a rule or cookbook change:

1. Add or edit files under `analyst/rules/` for new/updated rules, or under `docs/expression_rule_cookbook.md` (and related docs) for cookbook content.
2. Follow the existing YAML style and BRICK naming used in the cookbook.
3. In a PR, describe the fault the rule targets and any assumptions (e.g. Brick types, units). If it's from a standard (e.g. ASHRAE), note the reference.

48.8 Your First Code Contribution

- Look for issues labeled `good first issue` or `help wanted` (if we've added them).
- The codebase is Python 3.11+, with FastAPI (API), pandas/NumPy (rules), and Docker for the platform. See Getting Started and the README for setup.
- For rule or analyst changes, run the test suite and, if possible, the FDD loop locally with sample data before submitting a PR.

48.9 Improving the Documentation

Documentation lives in the `docs/` directory and is published at bbartling.github.io/open-fdd (Just the Docs theme). Improvements are welcome:

- Fix typos, clarify wording, or update steps.
- Add examples or how-tos (e.g. for the Expression Rule Cookbook, How-to Guides, or BACnet).
- Keep code blocks and links in sync with the codebase.

Building a PDF: To generate a single PDF of the docs (e.g. for offline use or to push to the repo), run `python3 scripts/build_docs_pdf.py`. Requires pandoc and either **weasyprint** (`pip install weasyprint`) or LaTeX. Output is `pdf/open-fdd-docs.pdf` by default; use `--output path` to change it. You can commit and push the PDF if you want it in the repo.

Open a PR with your changes; for large edits, an issue first can help align with maintainers.

48.10 Styleguides

- **Python:** We use Black for formatting. Run `black` on changed files before committing.
- **YAML:** Match the style in `analyst/rules/` and the expression rule cookbook (indentation, key order).
- **Markdown:** Use clear headings and lists; link to existing docs where relevant.

48.11 E2E frontend tests (Selenium)

End-to-end tests drive the **React frontend** with Selenium (no direct API calls from the test). They cover: delete-all-sites + reset, create site, import LLM payload (`scripts/demo_site_llm_payload.json`), and validation that **Plots** and **Weather** charts render and, when data exists, show data (not blank).

- **Install:** `pip install -e ".[e2e]"` (adds `selenium`, `webdriver-manager`).
- **Run:** Start the stack (frontend + API), then `python scripts/e2e_frontend_selenium.py`. Use `--frontend-url` if the app is not at `http://localhost:5173`, `--headed` to see the browser, `--only delete-all` or `--only charts` to run a subset.
- **Chart validation:** The test asserts that Recharts curve/path elements are present; if the BACnet scraper (or other source) has written timeseries data, it also asserts chart paths have data. See `scripts/e2e_frontend_selenium.py` and, for full flow, `docs/howto/e2e_selenium.md` (if present).

48.12 Git workflow (branch → PR → sync)

Create the branch **before** committing so that after the PR is merged, `git pull` on `main/master` fast-forwards and you avoid divergent-branch errors (protected `main/master`).

1. **Create a branch**

```
git checkout -b feature/short-name
```

2. **Stage, commit, and push**

```
git add .
git commit -m 'short note describing the change'
git push -u origin feature/short-name
```

3. **Open a PR manually on GitHub** from `feature/short-name` into `main` (or `master`). Create the PR in the GitHub UI; we do not use a CLI for this step.

4. **After the PR is merged** — fetch prune, switch to default branch, and pull to sync:

   ```
   git fetch --prune
   git checkout main
   git pull
   ```

   Use `master` instead of `main` if that is your default branch. If you ever committed on `main`/`master` before creating the branch, local and remote can diverge after the merge; run `git reset --hard origin/main` (or `origin/master`) to match the remote.

48.13 Commit Messages

- Use a clear, short summary line. Optionally add a body with details.
- Reference issues/PRs when relevant (e.g. `Fix bounds rule when input is all NaN (#123)`).

48.14 Join the Project Team

If you're interested in ongoing maintenance or a larger role, say so in an issue or reach out to the maintainers. We're especially interested in folks who can triage bugs, review rule contributions, or improve docs and the cookbook.

48.15 Attribution

This contributing guide was inspired by the contributing.md project. You don't need to pay to create or use a CONTRIBUTING file—it's just a markdown file in your repo that helps contributors and maintainers.

49 API Reference

50 API Reference

The Open-FDD platform exposes a **REST API** (port 8000) for CRUD, config, data model, bulk download, analytics, BACnet proxy, faults, and jobs. **Use the React frontend** (http://localhost:5173) for normal workflows—sites, points, config, faults, plots. Use the API when integrating scripts or cloud export.

Interactive docs: When the API is running, open **Swagger UI** at http://localhost:8000/docs or **ReDoc** at http://localhost:8000/redoc. Full **OpenAPI 3.1** spec: http://localhost:8000/openapi.json. When `OFDD_API_KEY` is set (e.g. in `stack/.env`), click **Authorize** in Swagger and paste the key so **Try it out** works.

50.1 High-level overview

Area	Purpose
CRUD	**Sites** — list, create, get, update, delete (cascade to equipment, points, timeseries, fault results). **Equipment** — list, create, get, update, delete (cascade to points). **Points** — list, create, get, update, delete (cascade to timeseries). Points can include `bacnet_device_id` and `object_identifier` so the BACnet scraper polls them.
Config	**GET/PUT /config** — platform config (rule interval, BACnet scrape interval, Open-Meteo, etc.) stored in the RDF graph. The React Config page uses these.
Data model	**GET /data-model/export** — BACnet discovery + DB points (for LLM tagging). **PUT /data-model/import** — bulk create/update points and equipment. **GET /data-model/ttl** — Brick + BACnet TTL. **POST /data-model/sparql** — run

Area	Purpose
	SPARQL. **POST /data-model/reset** — reset graph to DB-only.
Download	**GET/POST /download/csv** — timeseries CSV (wide or long). **GET /download/faults** — fault results (CSV or JSON). Excel-friendly; timestamps in UTC (Z).
Analytics	**GET /analytics/fault-summary**, **GET /analytics/fault-timeseries** — fault counts and time-series for charts. **GET /analytics/motor-runtime** — fan/VFD runtime (data-model driven). **GET /analytics/system/*** — host/container/disk metrics.
BACnet	**GET /bacnet/gateways** — list gateways. **POST /bacnet/server_hello**, **POST /bacnet/whois_range**, **POST /bacnet/point_discovery**, **POST /bacnet/point_discovery_to_graph** — proxy to diy-bacnet-server; discovery-to-graph feeds the data model. **POST /bacnet/write_point** — write value (audited; third-party integrations use this for audited writes).
Faults	**GET /faults/active**, **GET /faults/state**, **GET /faults/definitions** — active state, full state, definitions.
Jobs	**POST /jobs/bacnet/discovery**, **POST /jobs/fdd/run** — async BACnet discovery and FDD run. **GET /jobs/{job_id}** — status.
Run FDD	**POST /run-fdd** — trigger FDD run now. **GET /run-fdd/status** — last run.

Base URL: `http://localhost:8000` (use your host or IP when remote).
Auth: When `OFDD_API_KEY` is set, send `Authorization: Bearer <key>`; the React frontend does this when built with `VITE_OFDD_API_KEY`.

51 Appendix

52 Appendix

Technical deep dives and maintainer reference material.

Page	Description
API Reference	REST API at a glance: CRUD, config, data model, download, analytics, BACnet, faults, jobs. Full spec at Swagger /docs and /openapi.json.
Technical reference	Directory structure, environment variables, unit tests, BACnet scrape, bootstrap, database schema, LLM tagging workflow.
Developer guide	Front-end development, database schema (migrations, tables, cascade deletes), where to go next.

53 Frontend Evaluation

54 React frontend evaluation

This document summarizes the quality and feature coverage of the Open-FDD React frontend (CRUD UI), its alignment with the API and the Grafana cookbook, and recommendations for WebSockets and the legacy static UI.

54.1 Quality and stack

- **Stack:** React 19, TypeScript, Vite, TanStack Query, React Router, Recharts, Tailwind CSS, shadcn-style UI (Card, Table, Badge, Skeleton). Fits a modern SPA setup.
- **Structure:** Clear separation of pages, hooks (data + WebSocket), contexts (site, theme), and shared UI. API types in `src/types/api.ts`

match the OpenAPI schemas (Site, Equipment, Point, FaultState, FaultDefinition, etc.).
- **Auth:** Bearer token from `VITE_OFDD_API_KEY` is sent on all API requests and as `?token=` on the WebSocket; no auth UI (key is build-time env).
- **Missing pieces (added in this pass):** `src/lib/api.ts` (apiFetch with Bearer), `src/lib/csv.ts` (fetchCsv, parseLongCsv, pivotForChart for trending), and `src/lib/utils.ts` (cn, timeAgo, severityVariant) were not in the PR and are required for the app to build and run.

54.2 Feature coverage vs OpenAPI CRUD

API area	Frontend coverage
Sites	✅ List, select site; overview cards (equipment/points/faults per site). Create/update/delete not in UI (API-only or future).
Equipment	✅ List by site or all; table with point count and fault count.
Points	✅ List by site or all; table with external_id, equipment, brick_type, fdd_input, unit.
Faults	✅ Active faults list; fault definitions table + count (Grafana cookbook parity).
Trending	✅ Site-scoped point picker, date presets (24h, 7d, 30d, custom), line chart via POST /download/csv (long format) and pivot.
FDD status	✅ Last run in banner; run FDD is API-only (could add button).
Config, data-model, BACnet, jobs	❌ Not in UI; use Swagger or API.

So the frontend covers the main “operator” workflows: sites, equipment, points, active faults, fault definitions, and trending. Advanced flows (config, data-model export/import, BACnet discovery, jobs) stay in Swagger or the API.

54.3 Grafana cookbook parity

The Grafana SQL cookbook describes:

- **BACnet:** Variables (site, device, point), timeseries panel, BACnet + fault overlays.
  Frontend: Site selector + point picker + trending chart (same data via `/download/csv`). No separate "device" (bacnet_device_id) filter in the UI; points are chosen by site/equipment. Fault overlays on the chart could be a future enhancement.
- **Host stats:** Memory, load, swap, disk, containers.
  Frontend: No host-stats views; those remain in Grafana (or a future "System" page).
- **Weather:** Temp/RH/Dewpoint, wind, solar, cloud.
  Frontend: Covered indirectly: weather points (e.g. temp_f, rh_pct) appear in the point list and can be trended like any other point.
- **Faults:** Fault definitions table, fault definition count.
  Frontend: Implemented: "Fault definitions (N)" table (fault_id, name, category, severity, target equipment) and active faults table.

So the frontend is capable of the main cookbook workflows that are data-model/CRUD driven (sites, points, faults, trending). Grafana remains the place for raw SQL, host metrics, and pre-built dashboard JSON.

54.4 WebSockets: keep or remove?

- **Current use:** The React app uses `/ws/events` with token auth; on `fault.*`, `fdd.run`, `crud.*` it invalidates TanStack Query so lists and banners refresh. The HA integration optionally uses the same WebSocket for coordinator refresh.
- **Recommendation: keep WebSockets.** They give the React UI live updates without polling. If you later add MQTT for some integrations, you can keep WebSockets for the browser and use MQTT elsewhere; they are complementary (WS for real-time UI, MQTT for pub/sub or constrained clients). No need to remove the WebSocket feature.

54.5 Legacy static UI and BACnet tree

- The old config UI in `open_fdd/platform/static` (data-model tree Site → Equipment → Points, BACnet test/whois/discovery) was removed so the stack uses a single frontend: the React app, served from its own container and via Caddy.
- **BACnet tree / BAS-style tree:** The static UI had a tree that mirrored the data model and building automation. To get that in the React app, add a "Data model" or "Tree" page that:
  - Fetches `/sites`, then for each site `/equipment?site_id=`, then for each equipment `/points?equipment_id=`,
  - Renders a collapsible tree (sites → equipment → points) with the same behavior (expand/collapse, optional right-click delete).
  - Optionally add a "BACnet" section: server URL, Test (server_hello), Who-Is range, Point discovery, using the same API endpoints the static UI used (`/bacnet/server_hello`, `/bacnet/whois_range`, `/bacnet/point_discovery`). That would bring the legacy config + BACnet flows into the React app without serving static files from FastAPI.

54.6 Caddy and security

- With the React frontend in its own container, **Caddy** is the single entry point (e.g. :8088): basic auth, then proxy to API (and `/ws/*`) and to the frontend (`/*`). Optional Grafana at `/grafana` when started with `--with-grafana`. See Security and Caddy.

54.7 Summary

- **Quality:** Solid React/TS/Vite setup; missing `lib` (api, csv, utils) was added so the app builds and runs.
- **Features:** Good coverage of sites, equipment, points, faults, definitions, and trending; config/data-model/BACnet/jobs stay in API/Swagger.
- **Grafana cookbook:** Parity on fault definitions and trending; host stats and raw SQL stay in Grafana; optional to add fault overlays and a data-model/BACnet tree page later.
- **WebSockets:** Keep; they support real-time UI and HA. MQTT can be added alongside.

- **Static UI:** Removed; React is the primary UI, served by Caddy. BACnet tree and discovery can be reimplemented as a React page if desired.

55 Security

56 Security and Caddy Bootstrap

This document describes how to protect Open-FDD endpoints with **Caddy** (basic auth), which services are **unencrypted by default**, and **hardening** best practices. The project defaults to **non-TLS**; TLS (including self-signed certificates) is optional.

What is Caddy? A small web server in front of the API and the **React frontend**—one entry point (e.g. port 8088) with optional **basic auth** and **HTTPS**. Optional **Grafana** (when started with `--with-grafana`) is available at `/grafana`. By default Open-FDD does not run Caddy; this doc explains how to enable it for a protected entry point.

56.1 Architecture: frontend, API, and Caddy

- **React frontend** runs in its own container (port 5173 in dev; built and served via Caddy in production-style setups). It uses **Bearer token** auth against the Open-FDD API when `OFDD_API_KEY` is set: all requests (and the WebSocket at `/ws/events`) send `Authorization: Bearer <key>`. See Frontend API key (Bearer) below for where the key is sent.
- **API** (FastAPI) serves REST and WebSocket; when `OFDD_API_KEY` is set, it requires Bearer auth on all endpoints except `/health`, `/`, `/docs`, `/redoc`, `/openapi.json`, and `/app` (legacy static config UI).
- **Caddy** sits in front of both: it enforces **basic auth** (one login for the browser), then proxies to the API and frontend. When proxying to the API, Caddy adds the **X-Caddy-Auth** header (`header_up` in the Caddyfile) so the API accepts those requests without a Bearer token. So you sign in once with openfdd/xyz and the dashboard and API calls work without a second login. The web app can still send Bearer tokens when built with `VITE_OFDD_API_KEY`; when behind Caddy, the API trusts either Bearer or X-Caddy-Auth. **Best practice:** keep the frontend in its own container; do not serve the compiled

React app from FastAPI as static files. Caddy (or another reverse proxy) serves the frontend and proxies API/WebSocket.

56.2 Frontend API key (Bearer)

When `VITE_OFDD_API_KEY` is set (at frontend build time), the frontend sends it everywhere it talks to the backend: `apiFetch()` in `frontend/src/lib/api.ts` adds `Authorization: Bearer <key>` to all REST calls (sites, equipment, points, faults, config, FDD status, data model, etc.), `fetchCsv()` in `frontend/src/lib/csv.ts` adds the same header for CSV downloads, and the WebSocket in `frontend/src/hooks/use-websocket.ts` connects to `/ws/events?token=<key>`, which the backend accepts for WebSocket auth when an API key is configured.

56.3 Quick bootstrap with Caddy and basic auth

1. **Start the stack** (from repo root):

   ```
   ./scripts/bootstrap.sh
   ```

   This starts DB, API (8000), frontend (5173), BACnet server (8080), scrapers, FDD loop, and **Caddy on port 8088**. **Grafana is not started by default**; use `./scripts/bootstrap.sh --with-grafana` to include it (then access at `http://localhost:8088/grafana` or http://localhost:3000).

2. **Access through Caddy** (single entry point with basic auth):

   - **URL:** `http://localhost:8088`
   - **Default username:** `openfdd`
   - **Default password:** `xyz` (for local/testing only; change before any production or exposed use)

   The bundled Caddyfile routes API paths to the Open-FDD API, **WebSocket** `/ws/*` to the API, and `/*` to the **React frontend**. If Grafana was started with `--with-grafana`, it is available at `http://localhost:8088/grafana`. All routes require this one basic-auth credential. The React app then uses the **API key** (from `VITE_OFDD_API_KEY` at build time) for Bearer auth to the API.

3. **Without Caddy:** You can still use the services directly (no auth if `OFDD_API_KEY` is unset):
   - Frontend: http://localhost:5173
   - API: http://localhost:8000/docs
   - Grafana (if started): http://localhost:3000
     Do not expose these ports to untrusted networks without a reverse proxy and auth.

4. **Browser "Not secure" or "Your connection is not private":** Over plain HTTP (e.g. http://192.168.204.16:8088/), browsers show a "Not secure" or "connection not private" warning. That is expected without TLS. Use **http://** (not https://) so you are not blocked by a bad certificate. For production, add HTTPS (e.g. Caddy with TLS).

56.4 Caddyfile for protecting the entire API

The default **`stack/caddy/Caddyfile`** in the repo only proxies **port 80 → frontend** (no basic auth, API not behind Caddy). To protect the **entire** API (REST, WebSocket, docs) and frontend behind one entry point with basic auth, replace the Caddyfile with a block like the following. Use the same **secret** in `header_up X-Caddy-Auth` and in the API container env **`OFDD_CADDY_INTERNAL_SECRET`** so the API trusts requests that passed Caddy's basic auth.

```
# Listen on port 80 (or use :8088 and map 8088:8088 in docker-
compose).
:80 {
    # Optional: basic auth (generate hash: docker run --rm caddy:2
caddy hash-password --plaintext 'your_password')
    # basic_auth /* {
    #   openfdd <bcrypt_hash>
    # }

    # API: REST, Swagger, WebSocket. Set
OFDD_CADDY_INTERNAL_SECRET in the API container to match the value
below.
    handle /ws/* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /docs {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /redoc {
```

```
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /openapi.json {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /health {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /config* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /sites* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /equipment* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /points* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /faults* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /data-model* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /download* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /analytics* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /bacnet* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /jobs* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /capabilities* {
```

```
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }

    # Optional: Grafana when started with --with-grafana
    # handle /grafana/* {
    #   reverse_proxy grafana:3000
    # }

    # Frontend (React)
    handle /* {
        reverse_proxy frontend:5173
    }
}
```

Then restart Caddy and recreate the API so it has `OFDD_CADDY_INTERNAL_SECRET` set: `./scripts/bootstrap.sh --build caddy api` (from repo root). See Troubleshooting if the API returns 401 or the WebSocket fails.

56.5 Default password and how to change it

- The Caddyfile uses a **bcrypt hash** for the password. The default hash in the repo corresponds to password **`xyz`**.
- **Change the password** (recommended before production):

  1. Generate a new hash:

     ```
     docker run --rm caddy:2 caddy hash-password --plaintext 'your_secure_password'
     ```

  2. Open `stack/caddy/Caddyfile` and replace the hash on the `openfdd` line with the new output.

  3. Restart Caddy: `docker compose -f stack/docker-compose.yml restart caddy` (from repo root).

- **Advanced / multiple users:** Edit the `basic_auth` block in the Caddyfile and add more lines (one per `username hash`). Use `caddy hash-password` for each password. For SSO or advanced auth, consider Caddy's JWT or forward-auth and an IdP instead of basic auth.

56.6 Services: encryption and exposure

Service	Port	Encrypted by default?	Notes
API	8000	No (HTTP)	Put behind Caddy; do not expose directly. Bearer auth when `OFDD_API_KEY` is set.
Frontend	5173	No (HTTP)	React app; use Caddy to serve and protect. Sends Bearer token to API.
Grafana	3000	No (HTTP)	Optional (–with-grafana); use Caddy at /grafana or Grafana's own auth.
TimescaleDB	5432	No (plain PostgreSQL)	Not TLS by default. Keep on internal network only.
BACnet server	8080	No (HTTP API)	Host network; protect with firewall or Caddy on host.

- **Recommendation:** Expose only Caddy (e.g. 8088 or 443) to the building/remote network. Do not expose 5432, 8000, 5173, 3000, or 8080 to the internet. Use Caddy basic auth (and optionally TLS) for the frontend and API; keep the database and BACnet server behind

the firewall. The React frontend runs in its own container and is served by Caddy (or another reverse proxy), not as static files from the FastAPI process.

56.7 Hardening best practices

1. **Passwords**
   - Do not use default `openfdd` / `xyz` in production. Change via `caddy hash-password` and update the Caddyfile.
   - Use strong, unique passwords (or a secrets manager) for Grafana (`GF_SECURITY_ADMIN_PASSWORD`), Postgres (`POSTGRES_PASSWORD`), and Caddy basic auth.
2. **Network**
   - Run TimescaleDB and internal services on a private Docker network; bind DB to localhost or internal IP only if needed.
   - Restrict firewall so only Caddy's port (and optionally SSH) is reachable from outside.
3. **Reverse proxy**
   - Use Caddy in front of API and Grafana so one place enforces auth and (optionally) TLS.
   - Basic auth over plain HTTP is weak; prefer HTTPS when possible (see below).
4. **Principle of least privilege**
   - Run containers as non-root where possible; avoid sharing host network unless required (e.g. BACnet).
   - Limit Grafana sign-up and use strong admin credentials.
5. **Updates and hygiene**
   - Keep Caddy, Grafana, TimescaleDB, and the OS updated. Rotate credentials after compromise or periodically.

56.8 Throttling and rate limiting

56.8.1 1. No API rate limiting by default

Open-FDD does **not** rate-limit incoming HTTP requests to the API. There is no built-in cap on how often clients can call the API. If you need to limit how often external clients (e.g. a busy integration or cloud poller) can call the API, add rate limiting at the reverse proxy (e.g. Caddy with a rate-limit module) or with rate-limit middleware in front of the app.

56.8.2 2. Outbound: OT/building network is paced

The application **does** throttle its own outbound traffic to the building and OT network. We do not continuously hammer BACnet or other building systems. Load is paced by configuration **and by which points you scrape**:

Component	Config	Effect
BACnet scraper	`bacnet_scrape_interval_min` (e.g. 5)	Polls points on a fixed interval (e.g. every 5 minutes), not in a burst.
BACnet scraper	**Data model or CSV**	The scraper polls only the points it is configured with: **by default, points in the data model** that have `bacnet_device_id` and `object_identifier` (e.g. added via CRUD or after **POST /bacnet/point_discovery_to_graph** and data-model export/import). If none exist, it can fall back to a **curated CSV**. Throttling depends on **how many points** are defined (in the DB or in the CSV). Best practice: scrape only the points needed for FDD and HVAC health. See BACnet overview.
FDD rule loop	`rule_interval_hours` (e.g. 3)	Runs fault detection on a schedule (e.g. every 3 hours); each run pulls data from the DB, not from BACnet.
Weather scraper	`open_meteo_interval_hours` (e.g. 24)	Fetches weather once per interval (e.g. daily).

So outbound load on the OT network is predictable and tunable. **Define only the points you need** (in the data model via CRUD or point_discovery_to_graph + export/import, or in a curated CSV), then adjust intervals via `OFDD_*` environment variables. See Configuration and BACnet overview.

56.8.3 3. Inbound: rate limiting at the reverse proxy (e.g. Caddy)

If you need to **throttle incoming traffic** to the API—for example to protect the API and OT network from aggressive polling, misconfigured integrators, or abuse—enforce rate limiting at the reverse proxy or with middleware. Open-FDD does not implement this itself; use Caddy (with a rate-limit module), nginx, or application middleware.

Using Caddy for rate limiting

Caddy can throttle requests per client (e.g. per IP or per authenticated user) when configured with a rate-limiting capability. The standard Caddy build does not include an HTTP rate-limit directive; you add it by building Caddy with a module (e.g. `xcaddy build --with github.com/mholt/caddy-ratelimit`) or by using a distribution that includes one. Once available, you configure a rate-limit zone in the Caddyfile (e.g. limit by `{remote_host}` or by a header), then apply it to the blocks that proxy to the API (and optionally Grafana). Typical choices are a cap per IP (e.g. 60 requests per minute) or per client identity when using auth. Exceeding the limit returns `429 Too Many Requests`; the client can retry after the window resets.

Example (conceptual; syntax depends on the module you use):

```
:8088 {
  rate_limit {
    zone api { key {remote_host} events 60 duration 1m }
  }
  handle /api/* {
    rate_limit api
    reverse_proxy api:8000
  }
  # ... basic_auth and other handle blocks
}
```

Adjust the zone (events per duration) to match your OT network and integration requirements. For official options and current syntax, see Caddy's module documentation and the documentation for the rate-limit module you build or install.

56.9 TLS (optional): self-signed certificates

The project **default is non-TLS**. If you want HTTPS in front of Caddy:

1. **Option A – Caddy automatic HTTPS (public hostname)**
   If you have a public hostname and port 80/443, change the Caddyfile address to your domain; Caddy will obtain and renew Let's Encrypt certs automatically.

2. **Option B – Self-signed certificate for Caddy**
   For internal or lab use:

   - Generate a self-signed cert (e.g. with `openssl`).
   - In the Caddyfile, use `tls /path/to/cert.pem /path/to/key.pem` and listen on `:443` (or another port).
   - Mount the cert and key into the Caddy container and point Caddyfile to those paths.
   - Browsers will show a certificate warning; accept or install the CA for your environment.

Example Caddyfile snippet for self-signed (conceptual):

```
https://0.0.0.0:8443 {
  tls /etc/caddy/cert.pem /etc/caddy/key.pem
  basic_auth /* { ... }
  # same handle/reverse_proxy blocks as :8088
}
```

Generate self-signed (host):

```
        openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -
days 365 -nodes -subj "/CN=openfdd.local"
```

Mount `cert.pem` and `key.pem` into the Caddy container and reference them in `tls`.

56.10 Troubleshooting

API returns 401 (Unauthorized) after you pass Basic auth in the browser

The API trusts requests that carry the internal header `X-Caddy-Auth` (set by Caddy after Basic auth). The API only checks this when the env var `OFDD_CADDY_INTERNAL_SECRET` is set in the API container and matches the

value Caddy sends. If you added or changed this after the stack was already running, the API container may not have the variable. Recreate the API so it picks up the env:

```
cd stack && docker compose up -d --force-recreate api
```

Or from the repo root: `./scripts/bootstrap.sh --build api`. Then hard-refresh or use an incognito window and sign in again with Basic auth.

WebSocket to `/ws/events` fails or closes immediately

When access is through Caddy with Basic auth, the browser may not send credentials on the WebSocket upgrade. The API also accepts the same `X-Caddy-Auth` header on the WebSocket endpoint, so if Caddy is forwarding the request (after Basic auth), the WebSocket should work. Ensure the API has been recreated with `OFDD_CADDY_INTERNAL_SECRET` as above. If the WebSocket still fails (e.g. due to browser or proxy behavior), the UI will keep working; live updates will resume when the connection succeeds.

56.11 Security scanning (Trivy) — Phase 3

During development, use **Trivy** to scan container images and the repo for vulnerabilities and misconfigurations. Run it after building images, when changing Dockerfiles or dependencies, and optionally in CI. See **How-to: Trivy security scanning** for install, when to run, image and filesystem scan commands, and keeping docs up to date. High-level roadmap: NOTES.md (Phase 2: stack/DB/Caddy; Phase 3: Trivy in the loop).

56.12 Summary

- **Bootstrap:** Start the stack; use `http://localhost:8088` (or your Caddy port, e.g. 80) with user `openfdd` and default password `xyz` (change before production).
- **Passwords:** Change default by running `caddy hash-password` and updating the Caddyfile; use strong passwords for Grafana and Postgres.
- **Unencrypted by default:** API, Grafana, TimescaleDB, and BACnet API are plain HTTP/TCP; protect them with network isolation and Caddy (and optional TLS).

- **Hardening:** Strong passwords, expose only Caddy, keep DB and internal services off the public internet, keep software updated.
- **Throttling:** (1) No API rate limiting by default. (2) Outbound traffic to the OT/building network is paced (BACnet scrape, FDD loop, weather intervals). (3) To throttle incoming API traffic, use the reverse proxy (e.g. Caddy with a rate-limiting module) or middleware.
- **TLS:** Optional; default is non-TLS. Add self-signed or Let's Encrypt via Caddy when you need HTTPS.